HỌC PHẦN AN TOÀN VÀ BẢO MẬT THÔNG TIN

HƯỚNG DẪN THỰC HÀNH TỪNG BƯỚC

LAB 1

MÃ HÓA ĐỐI XỨNG CỔ ĐIỂN

Java Swing (NetBeans GUI Builder) – Maven – Microsoft SQL Server

Họ và tên: ..... MSSV: .....

Lớp: ..... Nhóm: ..... Ngày thực hành: ...../...../.....

Mục lục

TỔNG QUAN 4
LƯU Ý: nếu chưa cài JDK 17, NetBeans, SQL Server Express + SSMS (cổng 14333). Mật khẩu tài khoản SQL của mọi lab là LabUser@2026. Thì phải cài. 4
Kết quả của bài lab 4
PHẦN 1. CHUẨN BỊ PROJECT 5
1.1. Tạo project Maven `LAB1_MaHoaCoDien` 5
1.2. Làm quen màn hình thiết kế giao diện (NetBeans GUI Builder) 6
Năm thao tác dùng trong mọi bài 8
Thuộc tính chung của cửa sổ (làm cho MỌI form trừ frm_Menu) 8
1.3. Tạo lớp FileTextIO (dùng chung cho nút Open/Save của mọi form) 9
PHẦN 2. SÁU THUẬT TOÁN CƠ BẢN (BÀI 1.1 – 1.6) 11
BÀI 1.1. MẬT MÃ CAESAR 11
A. Hiểu thuật toán 11
B. Tạo lớp CaesarCipher 11
C. Thiết kế form frm_Caesar 12
D. Viết mã cho các nút 13
E. Chạy và kiểm tra 14
BÀI 1.2. MẬT MÃ VIGENÈRE 15
A. Hiểu thuật toán 15
B. Tạo lớp VigenereCipher 16
C. Thiết kế form frm_Vigenere 16
D. Viết mã cho các nút 17
E. Chạy và kiểm tra 18
BÀI 1.3. MẬT MÃ RAIL FENCE (HÀNG RÀO) 19
A. Hiểu thuật toán 19
B. Tạo lớp RailFenceCipher 19
C. Thiết kế form frm_RailFence 20
D. Viết mã cho các nút 21
E. Chạy và kiểm tra 22
BÀI 1.4. MẬT MÃ PLAYFAIR 23
A. Hiểu thuật toán 23
B. Tạo lớp PlayfairCipher 23
C. Thiết kế form frm_Playfair 25
D. Viết mã cho các nút 26
E. Chạy và kiểm tra 27
BÀI 1.5. MẬT MÃ HOÁN VỊ CỘT (TRANSPOSITION) 28
A. Hiểu thuật toán 28
B. Tạo lớp TranspositionCipher 28

C. Thiết kế form frm_Transposition 30
D. Viết mã cho các nút 30
E. Chạy và kiểm tra 31
BÀI 1.6. MẬT MÃ AUTOKEY 32
A. Hiểu thuật toán 32
B. Tạo lớp AutokeyCipher 32
C. Thiết kế form frm_Autokey 33
D. Viết mã cho các nút 34
E. Chạy và kiểm tra 34
PHẦN 3. BÀI NÂNG CAO 01 – CHƯƠNG TRÌNH TỔNG HỢP VÀ LƯU LỊCH SỬ SQL SERVER 36
3.1. Tạo lớp CipherTongHop 36
3.2. Tạo lớp kết nối DbConfig và HistoryDao 37
3.3. Thiết kế form frm_TongHop 39
Viết mã cho form frm_TongHop 41
Kiểm thử đủ 6 thuật toán 42
3.4. Cài đặt và cấu hình Microsoft SQL Server 42
a) Cài SQL Server Express và SSMS 43
b) Bật giao thức TCP/IP và đặt cổng 14333 43
c) Bật chế độ đăng nhập SQL Server (Mixed Mode) 43
Chạy các script tạo database và tài khoản 44
3.5. Tạo form frm_KetNoiSQL và frm_LichSu 45
Form frm_KetNoiSQL 45
Form frm_LichSu 46
Nối hai nút còn lại vào frm_TongHop 48
3.6. Kiểm tra ghi và đọc lịch sử 48
PHẦN 4. BÀI NÂNG CAO 02 – PHÂN TÍCH (TẤN CÔNG) MẬT MÃ CỔ ĐIỂN 50
A. Ba kỹ thuật sẽ cài đặt 50
B. Tạo lớp AttackDemos 50
C. Thiết kế form frm_TanCong 51
D. Các thí nghiệm bắt buộc (ghi kết quả vào báo cáo) 53
E. Nhận xét cần viết trong báo cáo 55
PHẦN 5. TẠO MENU CHÍNH VÀ CHẠY TOÀN BỘ CHƯƠNG TRÌNH 56

TỔNG QUAN

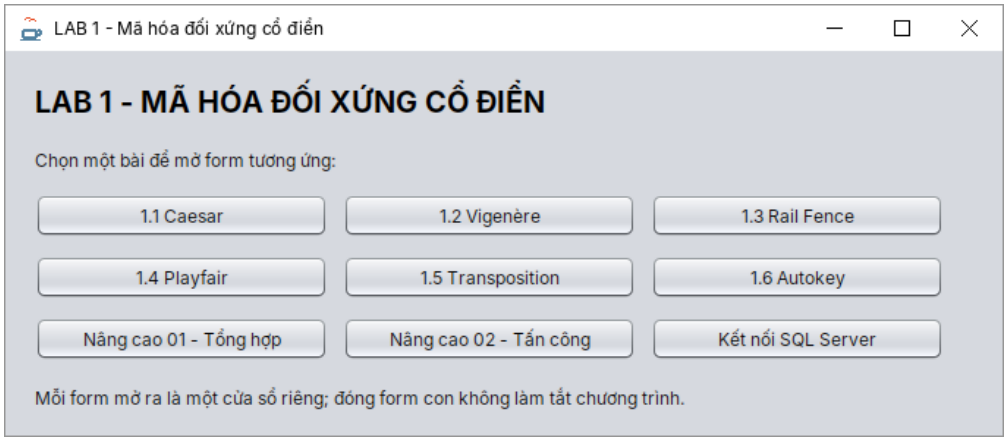
- Làm **lần lượt từ trên xuống**. Mỗi mục được chia thành các **Bước 1, 2, 3...**; làm xong bước này mới sang bước sau.
- Tên tệp, tên biến, mã lệnh hoặc giá trị phải gõ **chính xác từng ký tự** (phân biệt hoa/thường).
- Đoạn mã trong khung xám: chọn từ dòng đầu tới dòng cuối, **Copy (Ctrl+C)** rồi **Paste (Ctrl+V)** vào đúng vị trí được chỉ.
- Bảng điều khiển của mỗi form ghi đúng **loại điều khiển trên Palette**, **tên biến** và **thuộc tính**. Vị trí/kích thước chỉ cần gần giống ảnh mẫu.

LƯU Ý: nếu chưa cài JDK 17, NetBeans, SQL Server Express + SSMS (cổng 14333). Mật khẩu tài khoản SQL của mọi lab là LabUser@2026. Thì phải cài.

Phần mềm	Cách kiểm tra	Kết quả đúng
JDK 17 hoặc 21	Mở Command Prompt, gõ java -version	Thấy dòng openjdk version "17... hoặc "21...
Apache NetBeans (bản 20 trở lên)	Mở NetBeans → Help → About	Hiện số phiên bản, ví dụ 23 hoặc 25
Kết nối Internet (lần đầu)	Maven tải thư viện từ Internet khi Build	Output có BUILD SUCCESS
SQL Server Express + SSMS	Mở SSMS, kết nối localhost\SQLEXPRESS	Thấy cây Databases trong Object Explorer

Kết quả của bài lab

Hết bài, project LAB1_MaHoaCoDien có đúng các tệp sau trong package week_01



Hình 1.1. Cửa sổ chính frm_Menu khi hoàn thành bài (bấm Run Project – F6)

Bảng 1.1. Bộ dữ liệu kiểm thử bắt buộc (kết quả chương trình phải trùng từng ký tự)

Bài	Plaintext	Key	Ciphertext đúng
1.1 Caesar	HELLO	3	KHOOR
1.2 Vigenère	HELLO	KEY	RIJVS
1.3 Rail Fence	HELLO WORLD	3	HOREL OLLWD
1.4 Playfair	HELLOWORLD	KEYWORD	GYIZSCOKCFBU
1.5 Transposition	BAOMAT	2,3,1	AAOTBM
1.6 Autokey	HELLO	KEY	RIJSS

PHẦN 1. CHUẨN BỊ PROJECT

1.1. Tạo project Maven `LAB1_MaHoaCoDien`

Bước 1. Mở NetBeans → menu **File** → **New Project...** (phím tắt **Ctrl+Shift+N**).

Bước 2. Ở cột **Categories** chọn **Java with Maven**; cột **Projects** chọn **Java Application** → bấm **Next**.

Bước 3. Điền đúng các ô theo bảng dưới đây rồi bấm **Finish**.

Ô trên hộp thoại	Giá trị phải nhập	Ghi chú
Project Name	LAB1_MaHoaCoDien	Gõ đúng chữ hoa/thường
Project Location	C:\BaoMat	Thư mục không dấu, không khoảng trắng; dùng C:\BaoMat (NetBeans tự tạo nếu chưa có)
Group Id	vn.edu.lab	
Version	1.0-SNAPSHOT	Giữ mặc định
Package	week_01	Tên package bắt buộc của bài

Bước 4. NetBeans tạo sẵn một lớp có hàm main trong package week_01 (tên gần giống tên project). Nhấp phải lớp đó → Delete → OK (bài này dùng form frm_Menu làm cửa sổ chính).

Bước 5. Trong cửa sổ Projects, mở Project Files → pom.xml (nhấp đúp), dán toàn bộ nội dung dưới đây vào, bấm Ctrl+S để lưu.

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>vn.edu.lab</groupId>
    <artifactId>LAB1_MaHoaCoDien</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.release>17</maven.compiler.release>
        <!-- Lớp chạy khi bấm Run Project (F6) -->
        <exec.mainClass>week_01.frm_Menu</exec.mainClass>
    </properties>

    <dependencies>
        <!-- Microsoft JDBC Driver - kết nối SQL Server -->
        <dependency>
            <groupId>com.microsoft.sqlserver</groupId>
            <artifactId>mssql-jdbc</artifactId>
            <version>12.8.1.jre11</version>
        </dependency>
    </dependencies>
</project>
```

> **GHI CHÚ**
> Thư viện mssql-jdbc (Microsoft JDBC Driver) dùng ở Bài nâng cao 01 để kết nối SQL Server. Khai báo ngay từ đầu để chỉ phải Build một lần.

Dòng <exec.mainClass>week_01.frm_Menu</exec.mainClass> cho biết cửa sổ nào mở ra khi bấm Run Project (F6). Form frm_Menu được tạo ở phần cuối; trước đó hãy chạy từng form bằng Run File (Shift+F6).

Bước 6. Nhấp phải tên project → Clean and Build (Shift+F11). Lần đầu NetBeans sẽ tải thư viện từ Internet.

KẾT QUẢ ĐÚNG PHẢI THẤY

Cửa sổ **Output** (phía dưới) kết thúc bằng dòng BUILD SUCCESS.

Trong **Projects** có: LAB1_MaHoaCoDien → **Source Packages** → week_01 (đang trống) và mục **Dependencies** chứa các tệp .jar vừa tải.

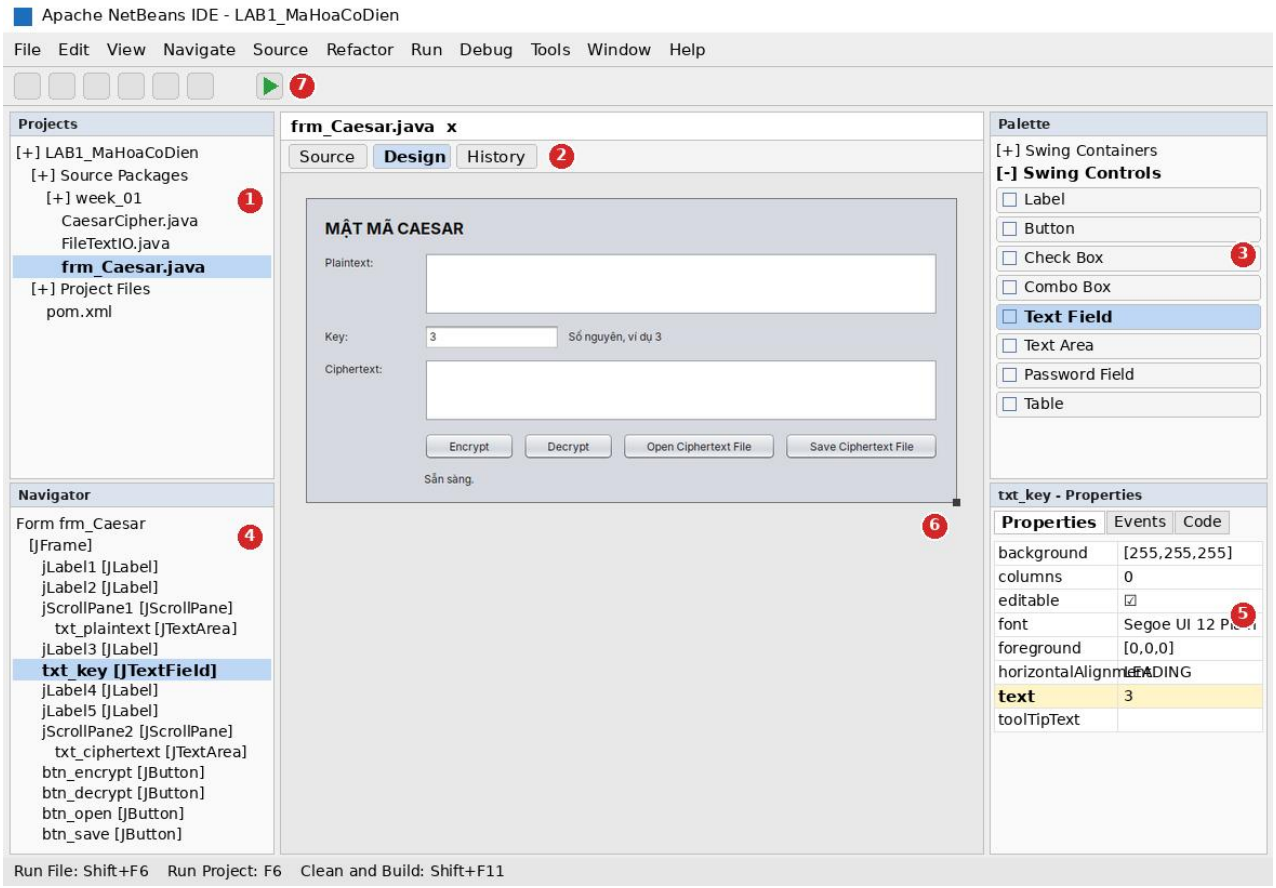
NẾU BỊ LỖI

BUILD FAILURE ... Could not resolve dependencies: máy chưa có Internet hoặc bị chặn proxy → kết nối mạng rồi Build lại.

release version 17 not supported: NetBeans đang dùng JDK cũ → Tools → Java Platforms thêm JDK 17/21, rồi nhấp phải project → Properties → Build → Compile → Java Platform chọn JDK mới.

1.2. Làm quen màn hình thiết kế giao diện (NetBeans GUI Builder)

Mọi giao diện trong bài đều là JFrame Form tạo bằng kéo-thả ở tab Design. NetBeans tự sinh mã giao diện (vùng mã màu xám trong tab Source - không sửa được, cũng không cần sửa). Ta chỉ làm 4 việc: (1) kéo điều khiển từ Palette, (2) đổi tên biến, (3) đặt thuộc tính, (4) viết mã cho sự kiện của nút.



Hình 1.2. Màn hình NetBeans khi thiết kế form frm_Caesar (các số ứng với bảng bên dưới)

Bảng 1.2. Các vùng làm việc trong Hình 1.2

Số	Vùng	Dùng để làm gì
1	Cửa sổ Projects	Nhấp phải vào package → New → JFrame Form... để tạo form; nhấp đúp tên tệp để mở.
2	Nút Source / Design	Design = kéo thả giao diện; Source = xem và viết mã Java (code sự kiện).
3	Palette (Swing Controls)	Nhấp chọn điều khiển (Label, Text Field, Text Area, Button, Combo Box, Table...) rồi nhấp vào form để đặt.
4	Navigator	Danh sách điều khiển trên form. Nhấp phải → Change Variable Name... để đổi tên biến (txt_..., btn_...).
5	Properties	Sửa thuộc tính của điều khiển đang chọn: text, editable, lineWrap, font, model...
6	Vùng thiết kế	Kéo cạnh/góc để chỉnh kích thước. Nhấp đúp một nút → NetBeans tạo hàm ...ActionPerformed và chuyển sang Source.
7	Nút Run	Run Project (F6) chạy lớp chính trong pom.xml; Run File (Shift+F6) chạy riêng form đang mở.

Năm thao tác dùng trong mọi bài

Bảng 1.3. Các thao tác cơ bản với GUI Builder

Thao tác	Cách làm trong NetBeans
A. Tạo JFrame Form	Nhấp phải package week_01 → New → JFrame Form... (hoặc New → Other... → Swing GUI Forms → JFrame Form) → gõ Class Name (ví dụ frm_Caesar) → Finish. Form trống mở ở tab Design.
B. Đặt điều khiển lên form	Ở Palette → Swing Controls, nhấp vào loại cần dùng (Label, Text Field...) rồi nhấp vào vị trí trên form. Kéo các ô vuông nhỏ ở cạnh để chỉnh kích thước. Đường gạch xanh hiện ra là đường căn lề - cứ thả theo đường đó. Text Area và Table được NetBeans tự bọc trong một jScrollPane - đây là điều bình thường.
C. Đổi tên biến	Nhấp phải điều khiển → Change Variable Name... → gõ tên mới (ví dụ txt_plaintext) → OK. Với Text Area/Table: nhấp phải vào bên trong ô (hoặc nhấp phải dòng txt_... trong cửa sổ Navigator), không phải viền jScrollPane. Label không cần đổi tên (trừ lbl_status, lbl_hint).
D. Đặt thuộc tính	Chọn điều khiển → cửa sổ Properties → sửa ô tương ứng: text (chữ hiển thị), editable (bỏ tích = chỉ đọc), lineWrap (tự xuống dòng), rows, font (bấm nút ... để chọn cỡ chữ), model (danh sách của Combo Box). Mẹo: với Label/Button có thể nhấp đúp lên chữ trên form để sửa text trực tiếp.
E. Viết mã cho nút	Ở tab Design nhấp đúp vào nút → NetBeans chuyển sang tab Source và tạo sẵn hàm private void btn_xxxActionPerformed(java.awt.event.ActionEvent evt) có dòng // TODO add your handling code here:. Xóa đúng dòng TODO, dán đoạn mã của bài vào chỗ đó. Đoạn mã trong tài liệu CHỈ là phần thân hàm nên không gõ lại dòng private void… và không gõ thêm dấu } (xem Hình 0.5 trong LAB0).

Thuộc tính chung của cửa sổ (làm cho MỌI form trừ frm_Menu)

Bước 1. Nhấp vào vùng trống của form (hoặc dòng [JFrame] trong Navigator) → trong Properties: ô defaultCloseOperation chọn DISPOSE; ô title gõ tiêu đề cửa sổ ghi trong từng bài.

Bước 2. Chuyển sang tab Source, ngay dưới dòng package ...; thêm dòng import javax.swing.JOptionPane; (để hiện hộp thông báo).

Bước 3. Trong hàm khởi tạo (constructor), ngay sau dòng initComponents(); thêm dòng setLocationRelativeTo(null); (cửa sổ hiện giữa màn hình).

Hai dòng cần thêm vào mỗi form

```
package week_01;

import javax.swing.JOptionPane;          // <-- thêm

public class frm_Caesar extends javax.swing.JFrame {

    public frm_Caesar() {
        initComponents();
        setLocationRelativeTo(null);       // <-- thêm
    }
    ...
```

> **GHI CHÚ**
> DISPOSE = đóng form này nhưng **không tắt** cả chương trình (các form được mở từ frm_Menu). Riêng frm_Menu để EXIT_ON_CLOSE (mặc định).
> Chạy thử một form: nhấp phải tệp frm_....java trong Projects → **Run File** (hoặc mở tệp rồi bấm **Shift+F6**).

1.3. Tạo lớp FileTextIO (dùng chung cho nút Open/Save của mọi form)

Bước 1. Nhấp phải package week_01 → New → Java Class... (nếu không thấy, chọn New → Other... → Java → Java Class).

Bước 2. Ô Class Name gõ FileTextIO → Finish. NetBeans mở tệp FileTextIO.java.

Bước 3. Bấm Ctrl+A, Delete để xóa nội dung mẫu, dán toàn bộ đoạn mã dưới đây, bấm Ctrl+S.

FileTextIO.java

```
package week_01;

import java.io.File;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import javax.swing.JFileChooser;
import javax.swing.JFrame;
import javax.swing.JOptionPane;
import javax.swing.JTextArea;
import javax.swing.filechooser.FileNameExtensionFilter;

/**
 * Mở và lưu tệp văn bản .txt (UTF-8) cho các JFrame Form.
 * Các hàm trả về đường dẫn tệp nếu thành công, trả về null nếu người dùng bấm Cancel hoặc bị lỗi.
 */
public final class FileTextIO {

    private FileTextIO() {
    }

    /** Hiện hộp thoại Open, đọc toàn bộ tệp và đưa nội dung vào ô target. */
    public static String openText(JFrame parent, JTextArea target) {
        JFileChooser chooser = new JFileChooser();
        chooser.setFileFilter(new FileNameExtensionFilter("Text file (*.txt)", "txt"));
        if (chooser.showOpenDialog(parent) != JFileChooser.APPROVE_OPTION) {
            return null;
        }
        File file = chooser.getSelectedFile();
        try {
```

```
            target.setText(Files.readString(file.toPath(), StandardCharsets.UTF_8).trim());
            return file.getAbsolutePath();
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(parent, "Không đọc được tệp: " + ex.getMessage(),
                    "Lỗi mở tệp", JOptionPane.ERROR_MESSAGE);
            return null;
        }
    }

    /** Hiện hộp thoại Save, ghi nội dung ô source ra tệp (tự thêm đuôi .txt nếu thiếu). */
    public static String saveText(JFrame parent, JTextArea source, String defaultName) {
        if (source.getText().isBlank()) {
            JOptionPane.showMessageDialog(parent, "Chưa có nội dung để lưu.");
            return null;
        }
        JFileChooser chooser = new JFileChooser();
        chooser.setSelectedFile(new File(defaultName));
        if (chooser.showSaveDialog(parent) != JFileChooser.APPROVE_OPTION) {
            return null;
        }
        File file = chooser.getSelectedFile();
        if (!file.getName().toLowerCase().endsWith(".txt")) {
            file = new File(file.getParentFile(), file.getName() + ".txt");
        }
        try {
            Files.writeString(file.toPath(), source.getText(), StandardCharsets.UTF_8);
            return file.getAbsolutePath();
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(parent, "Không lưu được tệp: " + ex.getMessage(),
                    "Lỗi lưu tệp", JOptionPane.ERROR_MESSAGE);
            return null;
        }
    }

    /** Hộp thoại Open chung: trả về nội dung tệp (UTF-8) hoặc null nếu Cancel/lỗi. */
    public static String openString(JFrame parent) {
        JFileChooser chooser = new JFileChooser();
        chooser.setFileFilter(new FileNameExtensionFilter("Text file (*.txt)", "txt"));
        if (chooser.showOpenDialog(parent) != JFileChooser.APPROVE_OPTION) {
            return null;
        }
        try {
            return Files.readString(chooser.getSelectedFile().toPath(), StandardCharsets.UTF_8);
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(parent, "Không đọc được tệp: " + ex.getMessage(),
                    "Lỗi mở tệp", JOptionPane.ERROR_MESSAGE);
            return null;
        }
    }

    /** Hộp thoại Save chung: ghi chuỗi content ra tệp, trả về đường dẫn hoặc null. */
    public static String saveString(JFrame parent, String content, String defaultName) {
        JFileChooser chooser = new JFileChooser();
        chooser.setSelectedFile(new File(defaultName));
        if (chooser.showSaveDialog(parent) != JFileChooser.APPROVE_OPTION) {
            return null;
        }
        File file = chooser.getSelectedFile();
        if (!file.getName().toLowerCase().endsWith(".txt")) {
            file = new File(file.getParentFile(), file.getName() + ".txt");
        }
        try {
            Files.writeString(file.toPath(), content, StandardCharsets.UTF_8);
            return file.getAbsolutePath();
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(parent, "Không lưu được tệp: " + ex.getMessage(),
                    "Lỗi lưu tệp", JOptionPane.ERROR_MESSAGE);
            return null;
        }
    }
}
```

- openText(this, txt_ciphertext): hiện hộp thoại chọn tệp, đọc tệp UTF-8 và đặt nội dung vào ô Ciphertext. Trả về đường dẫn tệp (hoặc null nếu bấm Cancel).

- saveText(this, txt_ciphertext, "caesar_ciphertext.txt"): hiện hộp thoại lưu, tự thêm đuôi .txt nếu thiếu.

PHẦN 2. SÁU THUẬT TOÁN CƠ BẢN (BÀI 1.1 – 1.6)

Mỗi bài gồm 4 việc giống nhau: **A. Hiểu thuật toán → B. Tạo lớp thuật toán → C. Thiết kế form → D. Viết mã cho các nút → E. Chạy kiểm thử.**

Bài 1.1 sẽ hướng dẫn chi tiết nhất; từ bài 1.2 trở đi bạn **sao chép form của bài 1.1** rồi chỉ sửa phần khác nhau.

BÀI 1.1. MẬT MÃ CAESAR

A. Hiểu thuật toán

- Mỗi chữ cái được **dịch đi k vị trí** trong bảng chữ cái (A=0, B=1, ..., Z=25), quay vòng sau Z.
- Mã hóa: **C = (P + k) mod 26**. Giải mã: **P = (C − k) mod 26**. Ký tự không phải chữ cái (khoảng trắng, số, dấu) giữ nguyên; chữ thường vẫn là chữ thường.

Bảng 1.4. Mã hóa tay HELLO với k = 3 → KHOOR

Bản rõ P	H	E	L	L	O
Số của P	7	4	11	11	14
P + 3	10	7	14	14	17
Bản mã C	K	H	O	O	R

B. Tạo lớp CaesarCipher

Bước 1. Nhấp phải package week_01 → New → Java Class... (nếu không thấy, chọn New → Other... → Java → Java Class).

Bước 2. Ô Class Name gõ CaesarCipher → Finish. NetBeans mở tệp CaesarCipher.java.

Bước 3. Bấm Ctrl+A, Delete để xóa nội dung mẫu, dán toàn bộ đoạn mã dưới đây, bấm Ctrl+S.

CaesarCipher.java

```
package week_01;

/** Mã Caesar: dịch chuyển A-Z/a-z; ký tự ngoài A-Z được giữ nguyên. */
public final class CaesarCipher {
    private CaesarCipher() { }

    public static String encrypt(String text, int key) {
        return transform(text, key);
    }

    public static String decrypt(String text, int key) {
        return transform(text, -key);
    }

    private static String transform(String text, int key) {
        StringBuilder result = new StringBuilder();
        int shift = Math.floorMod(key, 26);
        for (char c : text.toCharArray()) {
            char base;
            if (c >= 'A' && c <= 'Z') {
                base = 'A';
            } else if (c >= 'a' && c <= 'z') {
                base = 'a';
            } else {
                result.append(c);
                continue;
            }
            int original = c - base;
            int encoded = Math.floorMod(original + shift, 26);
            result.append((char) (base + encoded));
        }
        return result.toString();
    }
}
```

C. Thiết kế form frm_Caesar

Làm theo các thao tác A–D ở mục 1.2. Kết quả cần đạt như hình bên dưới.

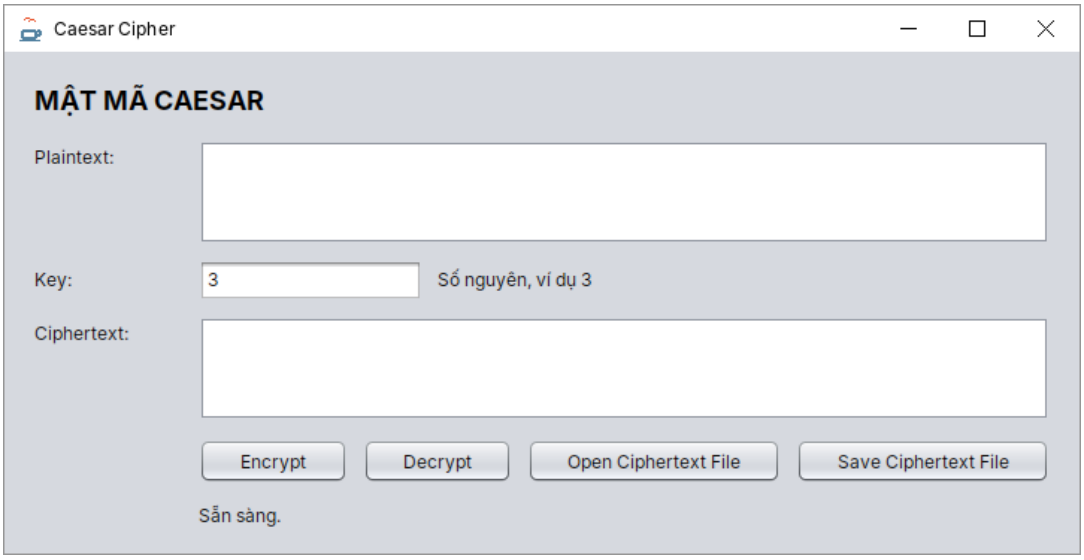
Bước 1.Tạo JFrame Form tên frm_Caesar (thao tác A). Đặt title = Caesar Cipher, defaultCloseOperation = DISPOSE.

Bước 2.Đặt các điều khiển theo thứ tự trong bảng (từ trên xuống, trái sang phải), đổi tên biến và đặt thuộc tính đúng như bảng.

Bảng 1.5. Điều khiển trên form frm_Caesar

#	Palette (Swing Controls)	Variable Name (đổi tên)	Thuộc tính cần đặt (Properties)	Cỡ gợi ý (R×C px)
1	Label	(giữ tên mặc định)	text = MẬT MÃ CAESAR font = Segoe UI 18 Bold	570×34
2	Label	(giữ tên mặc định)	text = Plaintext:	100×22
3	Text Area	txt_plaintext	lineWrap = tích rows = 3	570×70
4	Label	(giữ tên mặc định)	text = Key:	100×22
5	Text Field	txt_key	text = 3	150×28
6	Label	(giữ tên mặc định)	text = Số nguyên, ví dụ 3	410×22
7	Label	(giữ tên mặc định)	text = Ciphertext:	100×22
8	Text Area	txt_ciphertext	lineWrap = tích rows = 3	570×70
9	Button	btn_encrypt	text = Encrypt	100×30

#	Palette (Swing Controls)	Variable Name (đổi tên)	Thuộc tính cần đặt (Properties)	Cỡ gợi ý (R×C px)
10	Button	btn_decrypt	text = Decrypt	100×30
11	Button	btn_open	text = Open Ciphertext File	170×30
12	Button	btn_save	text = Save Ciphertext File	170×30
13	Label	lbl_status	text = Sẵn sàng.	570×22



Hình 1.3. Form frm_Caesar sau khi thiết kế (chạy thử bằng Shift+F6)

Bước 3. Bấm Ctrl+S, rồi nhấp phải tệp → Run File để xem thử: cửa sổ phải giống Hình 1.3 (bấm nút lúc này chưa có tác dụng).

D. Viết mã cho các nút

Bước 1. Thêm import javax.swing.JOptionPane; và setLocationRelativeTo(null); như mục 1.2 (Thuộc tính chung).

Bước 2. Nhấp đúp nút Encrypt (btn_encrypt) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_encryptActionPerformed

```
String plaintext = txt_plaintext.getText();
if (plaintext.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Hãy nhập Plaintext trước khi mã hóa.");
    return;
}
try {
    int key = Integer.parseInt(txt_key.getText().trim());
    txt_ciphertext.setText(CaesarCipher.encrypt(plaintext, key));
    lbl_status.setText("Đã mã hóa với khóa k = " + key + ".");
} catch (NumberFormatException ex) {
    JOptionPane.showMessageDialog(this, "Khóa Caesar phải là số nguyên, ví dụ 3.",
            "Khóa không hợp lệ", JOptionPane.ERROR_MESSAGE);
} catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(),
            "Dữ liệu không hợp lệ", JOptionPane.ERROR_MESSAGE);
}
```

Bước 3. Nhấp đúp nút Decrypt (btn_decrypt) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_decryptActionPerformed

```
String ciphertext = txt_ciphertext.getText();
if (ciphertext.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Hãy nhập Ciphertext trước khi giải mã.");
    return;
}
try {
    int key = Integer.parseInt(txt_key.getText().trim());
    txt_plaintext.setText(CaesarCipher.decrypt(ciphertext, key));
    lbl_status.setText("Đã giải mã với khóa k = " + key + ".");
} catch (NumberFormatException ex) {
    JOptionPane.showMessageDialog(this, "Khóa Caesar phải là số nguyên, ví dụ 3.",
            "Khóa không hợp lệ", JOptionPane.ERROR_MESSAGE);
} catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(),
            "Dữ liệu không hợp lệ", JOptionPane.ERROR_MESSAGE);
}
```

Bước 4. Nhấp đúp nút Open Ciphertext File (btn_open) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_openActionPerformed

```
String path = FileTextIO.openText(this, txt_ciphertext);
if (path != null) {
    lbl_status.setText("Đã mở tệp: " + path);
}
```

Bước 5. Nhấp đúp nút Save Ciphertext File (btn_save) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_saveActionPerformed

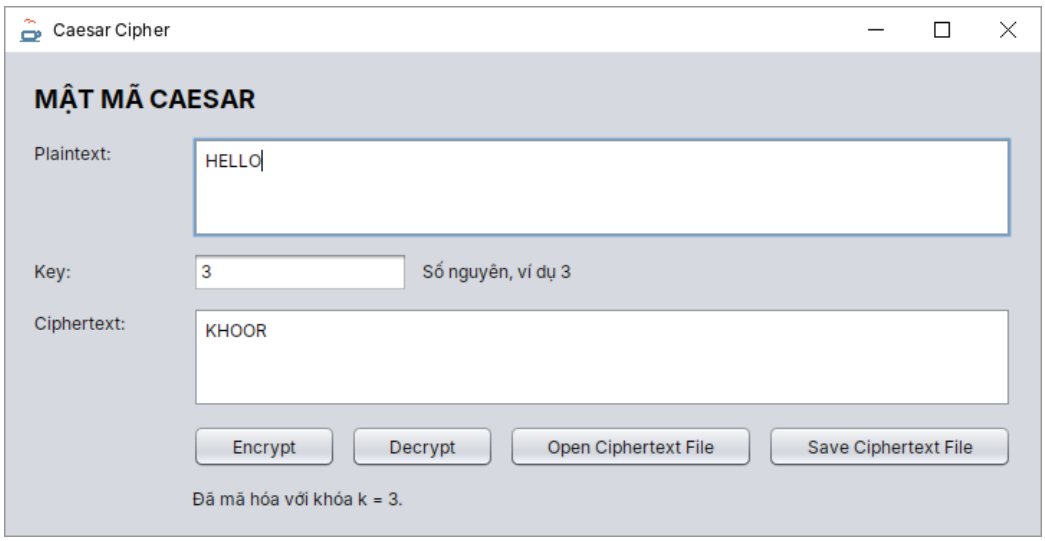
```
String path = FileTextIO.saveText(this, txt_ciphertext, "caesar_ciphertext.txt");
if (path != null) {
    lbl_status.setText("Đã lưu tệp: " + path);
}
```

Bước 6. Bấm Ctrl+S. Nếu NetBeans gạch đỏ dòng nào, rê chuột vào dòng đó để đọc lỗi và xem bảng lỗi ở cuối tài liệu.

E. Chạy và kiểm tra

Bước 1. Nhấp phải frm_Caesar.java → Run File (Shift+F6).

Bước 2. Nhập Plaintext HELLO, Key 3 → bấm Encrypt.

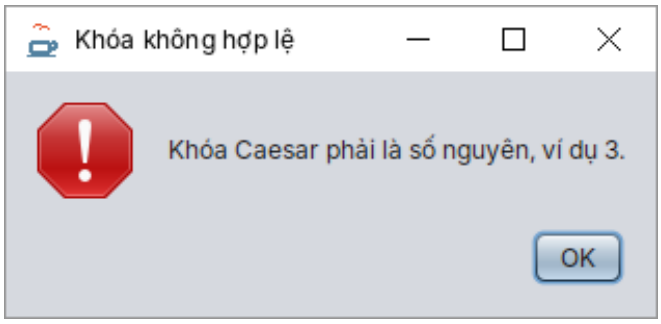


Hình 1.4. Kết quả mã hóa HELLO → KHOOR

Bước 3. Xóa ô Plaintext, bấm Decrypt → ô Plaintext hiện HELLO.

Bước 4. Bấm Save Ciphertext File → lưu caesar_ciphertext.txt vào thư mục bất kỳ. Xóa ô Ciphertext, bấm Open Ciphertext File → chọn lại tệp vừa lưu → bản mã hiện lại.

Bước 5. Thử khóa sai: Key = abc → bấm Encrypt → phải hiện hộp thông báo lỗi "Khóa Caesar phải là số nguyên, ví dụ 3.", chương trình không bị tắt.



Hình 1.5. Thông báo khi nhập khóa không phải số

BÀI 1.2. MẬT MÃ VIGENÈRE

A. Hiểu thuật toán

- Khóa là một từ (ví dụ KEY) được **lặp lại** cho đủ độ dài bản rõ; mỗi chữ bản rõ dịch theo chữ khóa tương ứng.
- Mã hóa: $\mathbf{C_i = (P_i + K_i) \bmod 26}$. Giải mã: $\mathbf{P_i = (C_i - K_i) \bmod 26}$. Khoảng trắng/dấu câu được giữ nguyên và **không** làm tiến khóa.

Bảng 1.6. HELLO + khóa KEY → RIJVS

Bản rõ	H (7)	E (4)	L (11)	L (11)	O (14)
Khóa lặp	K (10)	E (4)	Y (24)	K (10)	E (4)
Tổng mod 26	17	8	35 mod 26 = 9	21	18
Bản mã	R	I	J	V	S

B. Tạo lớp VigenereCipher

Bước 1. Nhấp phải package week_01 → New → Java Class... (nếu không thấy, chọn New → Other... → Java → Java Class).

Bước 2. Ô Class Name gõ VigenereCipher → Finish. NetBeans mở tệp VigenereCipher.java.

Bước 3. Bấm Ctrl+A, Delete để xóa nội dung mẫu, dán toàn bộ đoạn mã dưới đây, bấm Ctrl+S.

VigenereCipher.java

```
package week_01;

import java.util.Locale;

/** Vigenere: lặp khóa A-Z; dấu cách và dấu câu không làm tiến khóa. */
public final class VigenereCipher {
    private VigenereCipher() { }

    public static String encrypt(String text, String key) {
        return process(text, key, true);
    }

    public static String decrypt(String text, String key) {
        return process(text, key, false);
    }

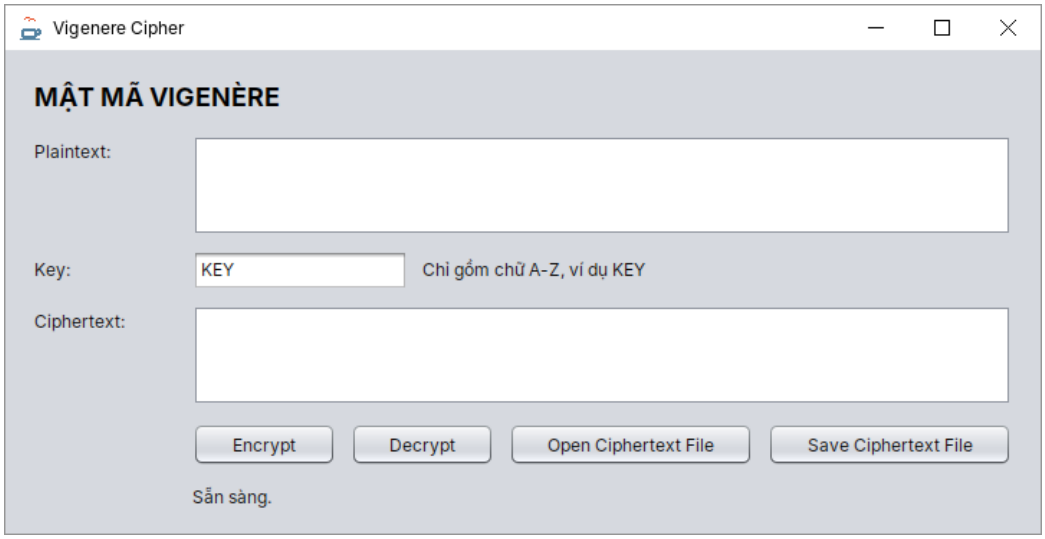
    private static String process(String text, String key, boolean encrypt) {
        String k = key.trim().toUpperCase(Locale.ROOT);
        if (!k.matches("[A-Z]+")) {
            throw new IllegalArgumentException("Khóa Vigenere chỉ gồm A-Z.");
        }
        StringBuilder result = new StringBuilder();
        int keyIndex = 0;
        for (char c : text.toCharArray()) {
            char base;
            if (c >= 'A' && c <= 'Z') {
                base = 'A';
            } else if (c >= 'a' && c <= 'z') {
                base = 'a';
            } else {
                result.append(c);
                continue;
            }
            int shift = k.charAt(keyIndex % k.length()) - 'A';
            keyIndex++;
            int delta = encrypt ? shift : -shift;
            result.append((char) (base + Math.floorMod(c - base + delta, 26)));
        }
        return result.toString();
    }
}
```

C. Thiết kế form frm_Vigenere

Bước 1. Trong Projects, nhấp phải frm_Caesar.java → Copy. Nhấp phải package week_01 → Paste → Refactor Copy... → ô New Name gõ frm_Vigenere → Refactor. NetBeans tạo frm_Vigenere.java (kèm tệp .form) và tự đổi tên lớp.

Bước 2.Mở frm_Vigenere.java ở tab Design, sửa 4 chỗ trong bảng sau (các điều khiển và tên biến giữ nguyên).

Sửa ở đâu	Giá trị mới
Thuộc tính title của JFrame	Vigenere Cipher
Label tiêu đề (jLabel1)	MẬT MÃ VIGENÈRE
Ô txt_key (thuộc tính text)	KEY
Label gợi ý cạnh ô Key (jLabel4)	Chỉ gồm chữ A-Z, ví dụ KEY



Hình 1.6. Form frm_Vigenere sau khi sửa

D. Viết mã cho các nút

Bước 1. Nhấp đúp nút Encrypt (btn_encrypt) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_encryptActionPerformed

```
String plaintext = txt  plaintext.getText();
if (plaintext.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Hãy nhập Plaintext trước khi mã hóa.");
    return;
}
try {
    String key = txt_key.getText().trim();
    txt_ciphertext.setText(VigenereCipher.encrypt(plaintext, key));
    lbl_status.setText("Đã mã hóa Vigenère với khóa " + key + ".");
} catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(),
            "Dữ liệu không hợp lệ", JOptionPane.ERROR_MESSAGE);
}
```

Bước 2. Nhấp đúp nút Decrypt (btn_decrypt) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_decryptActionPerformed

```
String ciphertext = txt_ciphertext.getText();
if (ciphertext.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Hãy nhập Ciphertext trước khi giải mã.");
    return;
}
try {
    String key = txt_key.getText().trim();
    txt_plaintext.setText(VigenereCipher.decrypt(ciphertext, key));
    lbl_status.setText("Đã giải mã Vigenère với khóa " + key + ".");
} catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(),
            "Dữ liệu không hợp lệ", JOptionPane.ERROR_MESSAGE);
}
```

Bước 3. Nhấp đúp nút Save Ciphertext File (btn_save) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_saveActionPerformed

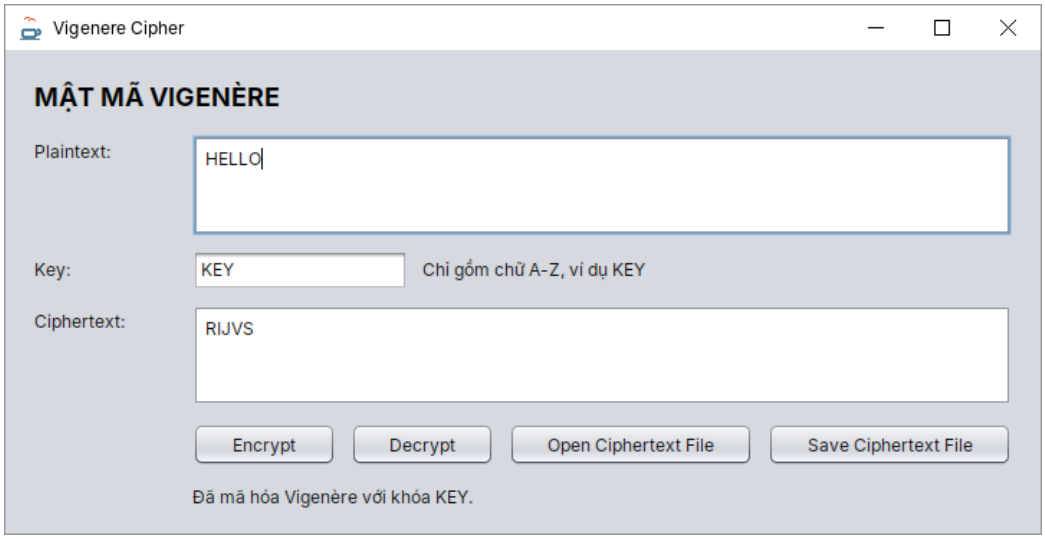
```
String path = FileTextIO.saveText(this, txt_ciphertext, "vigenere_ciphertext.txt");
if (path != null) {
    lbl_status.setText("Đã lưu tệp: " + path);
}
```

Bước 4. Bấm Ctrl+S. Nếu NetBeans gạch đỏ dòng nào, rê chuột vào dòng đó để đọc lỗi và xem bảng lỗi ở cuối tài liệu.

E. Chạy và kiểm tra

Bước 1. Nhấp phải frm_Vigenere.java → Run File (Shift+F6).

Bước 2. Nhập Plaintext HELLO, Key KEY → bấm Encrypt.



Hình 1.7. Kết quả mã hóa HELLO → RIJVS

Bước 3. Xóa ô Plaintext, bấm Decrypt → ô Plaintext hiện HELLO.

Bước 4. Bấm Save Ciphertext File → lưu vigenere_ciphertext.txt vào thư mục bất kỳ. Xóa ô Ciphertext, bấm Open Ciphertext File → chọn lại tệp vừa lưu → bản mã hiện lại.

Bước 5. Thử khóa sai: Key = K3Y → bấm Encrypt → phải hiện hộp thông báo lỗi "Khóa Vigenere chỉ gồm A-Z.", chương trình không bị tắt.

BÀI 1.3. MẬT MÃ RAIL FENCE (HÀNG RÀO)

A. Hiểu thuật toán

- Viết bản rõ theo **đường zíc-zắc** trên k hàng (đi xuống rồi đi lên), sau đó **đọc lần lượt từng hàng** từ trái sang phải. Khoảng trắng cũng là một ký tự.
- Giải mã: tính trước mỗi hàng có bao nhiêu ký tự, cắt bản mã thành từng hàng, rồi đọc lại theo đường zíc-zắc.

Bảng 1.7. "HELLO WORLD", 3 hàng: đọc hàng 1 "HOR" + hàng 2 "EL OL" + hàng 3 "LWD" = HOREL OLLWD

Vị trí	0	1	2	3	4	5	6	7	8	9	10
Hàng 1	H				O				R		
Hàng 2		E		L		␣		O		L	
Hàng 3			L				W				D

B. Tạo lớp RailFenceCipher

Bước 1. Nhấp phải package week_01 → New → Java Class... (nếu không thấy, chọn New → Other... → Java → Java Class).

Bước 2. Ô Class Name gõ RailFenceCipher → Finish. NetBeans mở tệp RailFenceCipher.java.

Bước 3. Bấm Ctrl+A, Delete để xóa nội dung mẫu, dán toàn bộ đoạn mã dưới đây, bấm Ctrl+S.

RailFenceCipher.java

```
package week_01;

/** Rail Fence: xếp các ký tự theo đường zigzag rồi đọc theo từng hàng. */
public final class RailFenceCipher {
    private RailFenceCipher() { }

    public static String encrypt(String text, int rails) {
        return process(text, rails, true);
    }

    public static String decrypt(String text, int rails) {
        return process(text, rails, false);
    }

    private static int[] railPattern(int size, int rails) {
        int[] pattern = new int[size];
        int row = 0;
        int direction = 1;
        for (int i = 0; i < size; i++) {
            pattern[i] = row;
            if (row == 0) direction = 1;
```

```
            else if (row == rails - 1) direction = -1;
            row += direction;
        }
        return pattern;
    }

    private static String process(String text, int rails, boolean encrypt) {
        if (rails < 2 || rails > 1000) {
            throw new IllegalArgumentException("Số hàng phải từ 2 đến 1000.");
        }
        int[] characters = text.codePoints().toArray();
        int length = characters.length;
        if (length <= 1 || rails >= length) return text;

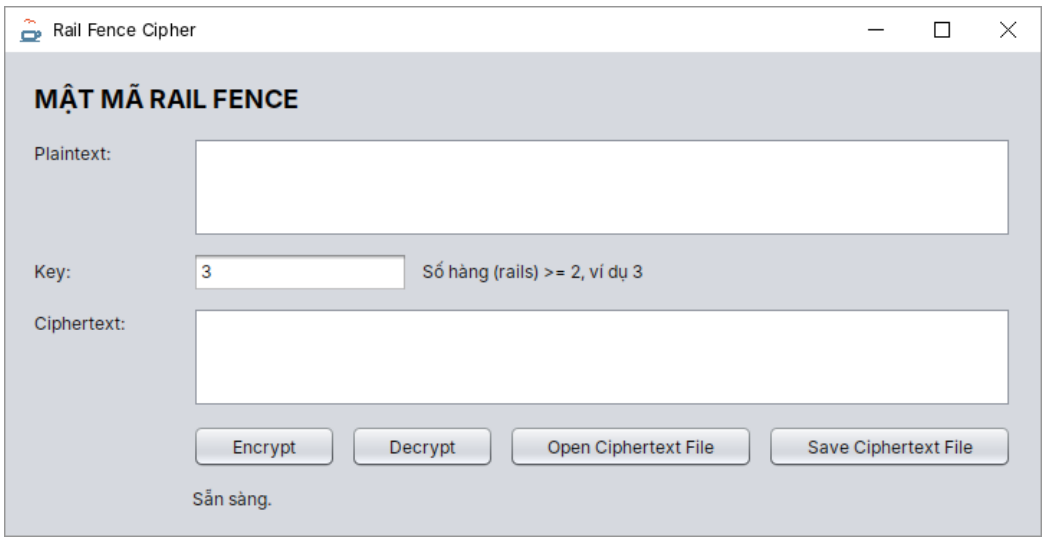
        int[] pattern = railPattern(length, rails);
        int[] result = new int[length];
        int offset = 0;
        if (encrypt) {
            for (int rail = 0; rail < rails; rail++) {
                for (int i = 0; i < length; i++) {
                    if (pattern[i] == rail) result[offset++] = characters[i];
                }
            }
        } else {
            int[] index = new int[length];
            for (int rail = 0; rail < rails; rail++) {
                for (int i = 0; i < length; i++) {
                    if (pattern[i] == rail) index[i] = offset++;
                }
            }
            for (int i = 0; i < length; i++) {
                result[i] = characters[index[i]];
            }
        }
        return new String(result, 0, result.length);
    }
}
```

C. Thiết kế form frm_RailFence

Bước 1. Trong Projects, nhấp phải frm_Caesar.java → Copy. Nhấp phải package week_01 → Paste → Refactor Copy... → ô New Name gõ frm_RailFence → Refactor. NetBeans tạo frm_RailFence.java (kèm tệp .form) và tự đổi tên lớp.

Bước 2. Mở frm_RailFence.java ở tab Design, sửa 4 chỗ trong bảng sau (các điều khiển và tên biến giữ nguyên).

Sửa ở đâu	Giá trị mới
Thuộc tính title của JFrame	Rail Fence Cipher
Label tiêu đề (jLabel1)	MẬT MÃ RAIL FENCE
Ô txt_key (thuộc tính text)	3
Label gợi ý cạnh ô Key (jLabel4)	Số hàng (rails) >= 2, ví dụ 3



Hình 1.8. Form frm_RailFence sau khi sửa

D. Viết mã cho các nút

Bước 1. Nhấp đúp nút Encrypt (btn_encrypt) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_encryptActionPerformed.

```
String plaintext = txt_plaintext.getText();
if (plaintext.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Hãy nhập Plaintext trước khi mã hóa.");
    return;
}
try {
    int rails = Integer.parseInt(txt_key.getText().trim());
    txt_ciphertext.setText(RailFenceCipher.encrypt(plaintext, rails));
    lbl_status.setText("Đã mã hóa với " + rails + " hàng.");
} catch (NumberFormatException ex) {
    JOptionPane.showMessageDialog(this, "Số hàng phải là số nguyên, ví dụ 3.",
            "Khóa không hợp lệ", JOptionPane.ERROR_MESSAGE);
} catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(),
            "Dữ liệu không hợp lệ", JOptionPane.ERROR_MESSAGE);
}
```

Bước 2. Nhấp đúp nút Decrypt (btn_decrypt) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_decryptActionPerformed

```
String ciphertext = txt_ciphertext.getText();
if (ciphertext.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Hãy nhập Ciphertext trước khi giải mã.");
    return;
}
try {
    int rails = Integer.parseInt(txt_key.getText().trim());
    txt_plaintext.setText(RailFenceCipher.decrypt(ciphertext, rails));
    lbl_status.setText("Đã giải mã với " + rails + " hàng.");
} catch (NumberFormatException ex) {
    JOptionPane.showMessageDialog(this, "Số hàng phải là số nguyên, ví dụ 3.",
            "Khóa không hợp lệ", JOptionPane.ERROR_MESSAGE);
} catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(),
            "Dữ liệu không hợp lệ", JOptionPane.ERROR_MESSAGE);
}
```

Bước 3. Nhấp đúp nút Save Ciphertext File (btn_save) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_saveActionPerformed.

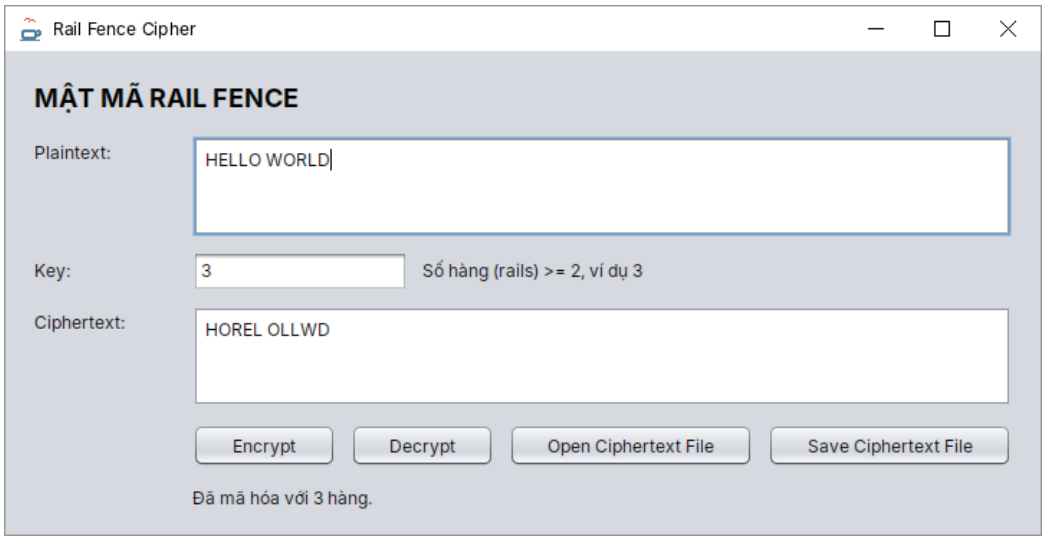
```
String path = FileTextIO.saveText(this, txt_ciphertext, "railfence_ciphertext.txt");
if (path != null) {
    lbl_status.setText("Đã lưu tệp: " + path);
}
```

Bước 4. Bấm Ctrl+S. Nếu NetBeans gạch đỏ dòng nào, rê chuột vào dòng đó để đọc lỗi và xem bảng lỗi ở cuối tài liệu.

E. Chạy và kiểm tra

Bước 1. Nhấp phải frm_RailFence.java → **Run File** (Shift+F6).

Bước 2. Nhập Plaintext HELLO WORLD, Key 3 → bấm **Encrypt**.



Hình 1.9. Kết quả mã hóa HELLO WORLD → HOREL OLLWD

Bước 3. Xóa ô Plaintext, bấm Decrypt → ô Plaintext hiện HELLO WORLD.

Bước 4. Bấm Save Ciphertext File → lưu railfence_ciphertext.txt vào thư mục bất kỳ. Xóa ô Ciphertext, bấm Open Ciphertext File → chọn lại tệp vừa lưu → bản mã hiện lại.

Bước 5. Thử khóa sai: Key = 1 → bấm Encrypt → phải hiện hộp thông báo lỗi "Số hàng phải từ 2 đến 1000.", chương trình không bị tắt.

BÀI 1.4. MẬT MÃ PLAYFAIR

A. Hiểu thuật toán

- **Ma trận khóa 5×5**: viết các chữ của khóa (bỏ chữ trùng) rồi các chữ còn lại của bảng chữ cái; **I và J dùng chung một ô**.
- **Chuẩn bị bản rõ**: bỏ khoảng trắng, đổi J → I, chia thành **cặp 2 chữ**; nếu một cặp có 2 chữ giống nhau thì chèn X vào giữa; thiếu chữ cuối thì thêm X.
- Với mỗi cặp: **cùng hàng** → lấy chữ bên phải; **cùng cột** → lấy chữ bên dưới; **khác hàng, khác cột** → lấy chữ ở góc còn lại của hình chữ nhật (cùng hàng với chữ đó).
- Giải mã làm ngược lại (bên trái / bên trên). Bản rõ giải ra còn chữ X đệm (HELXLOWORLDX) - đây là kết quả đúng.

Bảng 1.8. Ma trận khóa KEYWORD

	Cột 0	Cột 1	Cột 2	Cột 3	Cột 4
Hàng 0	K	E	Y	W	O
Hàng 1	R	D	A	B	C
Hàng 2	F	G	H	I/J	L
Hàng 3	M	N	P	Q	S
Hàng 4	T	U	V	X	Z

Bảng 1.9. HELLOWORLD → HE LX LO WO RL DX → GYIZSCOKCFBU

Cặp	HE	LX	LO	WO	RL	DX
Quy tắc	chữ nhật	chữ nhật	cùng cột	cùng hàng	chữ nhật	chữ nhật
Kết quả	GY	IZ	SC	OK	CF	BU

B. Tạo lớp PlayfairCipher

Bước 1. Nhấp phải package week_01 → New → Java Class... (nếu không thấy, chọn New → Other... → Java → Java Class).

Bước 2. Ô Class Name gõ PlayfairCipher → Finish. NetBeans mở tệp PlayfairCipher.java.

Bước 3. Bấm Ctrl+A, Delete để xóa nội dung mẫu, dán toàn bộ đoạn mã dưới đây, bấm Ctrl+S.

PlayfairCipher.java

```
package week_01;

import java.util.Locale;

/** Playfair 5x5: I/J chung ô, dùng X/Q làm ký tự đệm. */
public final class PlayfairCipher {
    private final char[][] matrix;

    public PlayfairCipher(String key) {
        this.matrix = makeMatrix(key);
    }

    public String encrypt(String text) {
        return process(text, true);
```

```
    }

    public String decrypt(String text) {
        return process(text, false);
    }

    public static String encrypt(String text, String key) {
        return new PlayfairCipher(key).encrypt(text);
    }

    public static String decrypt(String text, String key) {
        return new PlayfairCipher(key).decrypt(text);
    }

    public static String getKeyMatrix(String key) {
        StringBuilder display = new StringBuilder();
        for (char[] row : makeMatrix(key)) {
            for (char c : row) display.append(c).append(' ');
            display.append('\n');
        }
        return display.toString();
    }

    private static char[][] makeMatrix(String key) {
        String upper = key.trim().toUpperCase(Locale.ROOT);
        if (!upper.matches("[A-Z]+")) {
            throw new IllegalArgumentException("Khóa Playfair chỉ gồm A-Z.");
        }
        boolean[] used = new boolean[26];
        StringBuilder alphabet = new StringBuilder();
        for (char c : (upper + "ABCDEFGHIKLMNOPQRSTUVWXYZ").toCharArray()) {
            if (c == 'J') c = 'I';
            if (!used[c - 'A']) {
                used[c - 'A'] = true;
                alphabet.append(c);
            }
        }
        char[][] output = new char[5][5];
        for (int i = 0; i < 25; i++) {
            output[i / 5][i % 5] = alphabet.charAt(i);
        }
        return output;
    }

    private static String normalize(String text) {
        StringBuilder clean = new StringBuilder();
        for (char c : text.toUpperCase(Locale.ROOT).toCharArray()) {
            if (c >= 'A' && c <= 'Z') clean.append(c == 'J' ? 'I' : c);
        }
        if (clean.length() == 0) {
            throw new IllegalArgumentException("Văn bản Playfair cần chữ A-Z.");
        }
        return clean.toString();
    }

    private static String makePairs(String normalized) {
        StringBuilder pairs = new StringBuilder();
        int i = 0;
        while (i < normalized.length()) {
            char first = normalized.charAt(i++);
            char second;
            if (i >= normalized.length() || first == normalized.charAt(i)) {
                second = first == 'X' ? 'Q' : 'X';
            } else {
                second = normalized.charAt(i++);
            }
            pairs.append(first).append(second);
        }
        return pairs.toString();
    }

    private int[] locate(char c) {
        for (int row = 0; row < 5; row++) {
            for (int col = 0; col < 5; col++) {
                if (matrix[row][col] == c) return new int[] {row, col};
            }
```

```
        }
        throw new IllegalArgumentException("Ký tự không có trong bảng khóa.");
    }

    private String process(String text, boolean encrypt) {
        String data = normalize(text);
        if (encrypt) data = makePairs(data);
        else if (data.length() % 2 != 0) {
            throw new IllegalArgumentException("Bản mã Playfair phải có độ dài chẵn.");
        }
        int step = encrypt ? 1 : 4;
        StringBuilder output = new StringBuilder();
        for (int i = 0; i < data.length(); i += 2) {
            int[] a = locate(data.charAt(i));
            int[] b = locate(data.charAt(i + 1));
            if (a[0] == b[0]) {
                output.append(matrix[a[0]][(a[1] + step) % 5]);
                output.append(matrix[b[0]][(b[1] + step) % 5]);
            } else if (a[1] == b[1]) {
                output.append(matrix[(a[0] + step) % 5][a[1]]);
                output.append(matrix[(b[0] + step) % 5][b[1]]);
            } else {
                output.append(matrix[a[0]][b[1]]);
                output.append(matrix[b[0]][a[1]]);
            }
        }
        return output.toString();
    }
}
```

C. Thiết kế form frm_Playfair.

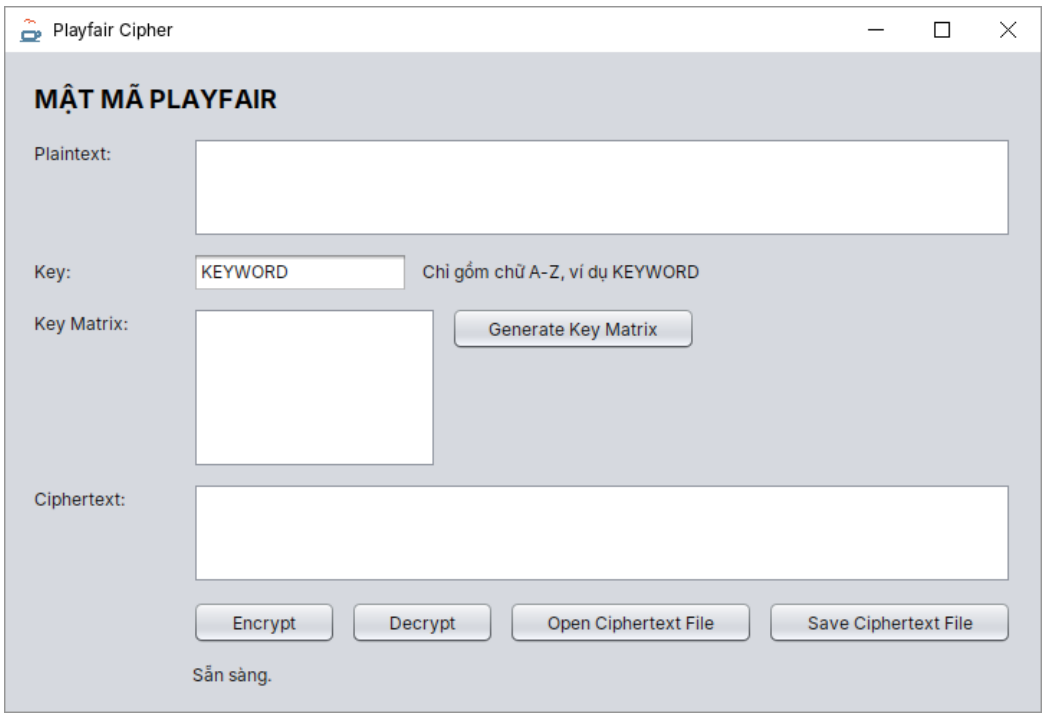
Bước 1. Trong Projects, nhấp phải frm_Caesar.java → Copy. Nhấp phải package week_01 → Paste → Refactor Copy... → ô New Name gõ frm_Playfair → Refactor. NetBeans tạo frm_Playfair.java (kèm tệp .form) và tự đổi tên lớp.

Bước 2. Mở frm_Playfair.java ở tab Design, sửa 4 chỗ trong bảng sau (các điều khiển và tên biến giữ nguyên).

Sửa ở đâu	Giá trị mới
Thuộc tính title của JFrame	Playfair Cipher
Label tiêu đề (jLabel1)	MẬT MÃ PLAYFAIR
Ô txt_key (thuộc tính text)	KEYWORD
Label gợi ý cạnh ô Key (jLabel4)	Chỉ gồm chữ A-Z, ví dụ KEYWORD

Bước 3. Kéo phần dưới của form xuống để có chỗ trống giữa ô Key và ô Ciphertext, rồi thêm 2 điều khiển mới:

Palette	Variable Name	Thuộc tính
Label	(giữ tên mặc định)	text = Key Matrix:
Text Area	txt_matrix	editable = bỏ tích; lineWrap = không tích; rows = 5; columns = 9; font = Monospaced 14 Bold; cỡ ≈ 170×112
Button	btn_matrix	text = Generate Key Matrix



Hình 1.10. Form frm_Playfair sau khi sửa

D. Viết mã cho các nút

Bước 1. Nhấp đúp nút Encrypt (btn_encrypt) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_encryptActionPerformed.

```
String plaintext = txt_plaintext.getText();
if (plaintext.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Hãy nhập Plaintext trước khi mã hóa.");
    return;
}
try {
    String key = txt_key.getText().trim();
    txt_ciphertext.setText(PlayfairCipher.encrypt(plaintext, key));
    lbl_status.setText("Đã mã hóa Playfair (bỏ dấu cách, J -> I, chèn X).");
} catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(),
            "Dữ liệu không hợp lệ", JOptionPane.ERROR_MESSAGE);
}
```

Bước 2. Nhấp đúp nút Decrypt (btn_decrypt) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_decryptActionPerformed

```
String ciphertext = txt_ciphertext.getText();
if (ciphertext.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Hãy nhập Ciphertext trước khi giải mã.");
    return;
}
try {
    String key = txt_key.getText().trim();
    txt_plaintext.setText(PlayfairCipher.decrypt(ciphertext, key));
    lbl_status.setText("Đã giải mã Playfair (còn ký tự đệm X).");
} catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(),
            "Dữ liệu không hợp lệ", JOptionPane.ERROR_MESSAGE);
}
```

Bước 3. Nhấp đúp nút Save Ciphertext File (btn_save) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_saveActionPerformed

```
String path = FileTextIO.saveText(this, txt_ciphertext, "playfair_ciphertext.txt");
if (path != null) {
    lbl_status.setText("Đã lưu tệp: " + path);
}
```

Bước 4. Nhấp đúp nút Generate Key Matrix (btn_matrix) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_matrixActionPerformed

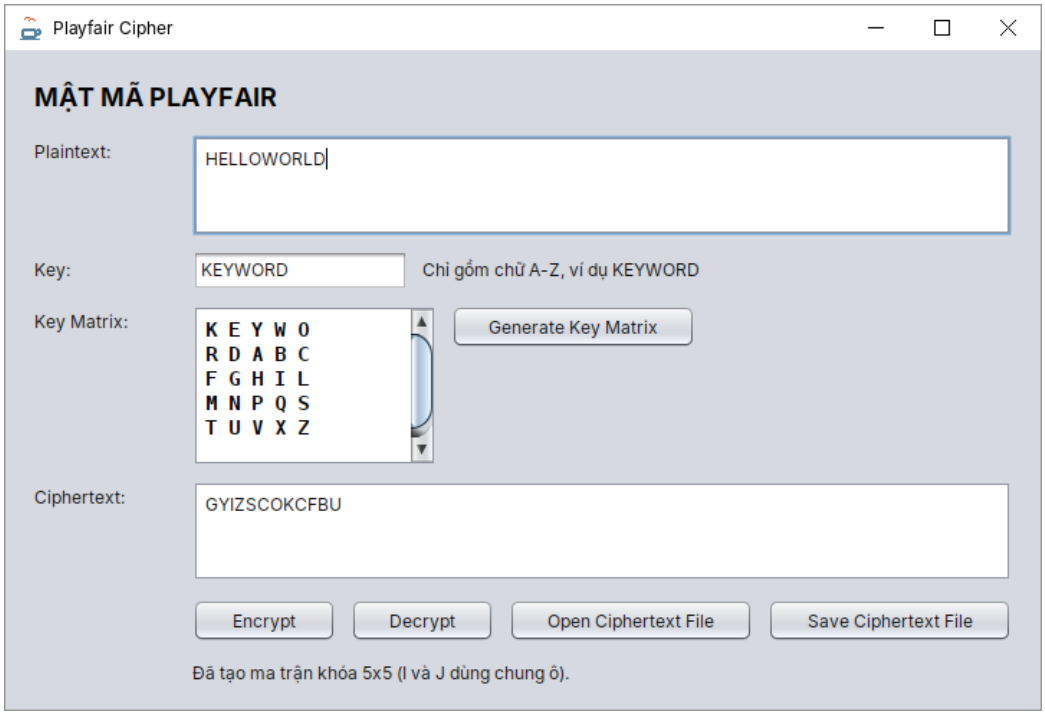
```
try {
    txt_matrix.setText(PlayfairCipher.getKeyMatrix(txt_key.getText()));
    txt_matrix.setCaretPosition(0);
    lbl_status.setText("Đã tạo ma trận khóa 5x5 (I và J dùng chung ô).");
} catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(),
            "Khóa không hợp lệ", JOptionPane.ERROR_MESSAGE);
}
```

Bước 5. Bấm Ctrl+S. Nếu NetBeans gạch đỏ dòng nào, rê chuột vào dòng đó để đọc lỗi và xem bảng lỗi ở cuối tài liệu.

E. Chạy và kiểm tra

Bước 1. Nhấp phải frm_Playfair.java → Run File (Shift+F6).

Bước 2. Nhập Plaintext HELLOWORLD, Key KEYWORD → bấm Encrypt → bấm Generate Key Matrix.



Hình 1.11. Kết quả mã hóa HELLOWORLD → GYIZSCOKCFBU

Bước 3.Xóa ô Plaintext, bấm Decrypt → ô Plaintext hiện HELXLOWORLDX (có chữ X đệm - đúng thuật toán).

Bước 4.Bấm Save Ciphertext File → lưu playfair_ciphertext.txt vào thư mục bất kỳ. Xóa ô Ciphertext, bấm Open Ciphertext File → chọn lại tệp vừa lưu → bản mã hiện lại.

Bước 5.Thử khóa sai: Key = KEY WORD → bấm Encrypt → phải hiện hộp thông báo lỗi "Khóa Playfair chỉ gồm A-Z.", chương trình không bị tắt.

BÀI 1.5. MẬT MÃ HOÁN VỊ CỘT (TRANSPOSITION)

A. Hiểu thuật toán

- Khóa là **thứ tự đọc các cột**, ví dụ 2,3,1 = có 3 cột, đọc cột 2 trước, rồi cột 3, cuối cùng cột 1.
- Mã hóa: viết bản rõ **theo hàng** vào bảng có số cột = độ dài khóa, sau đó **đọc theo cột** theo thứ tự khóa. Giải mã: điền ngược theo cột rồi đọc theo hàng.
- Khóa phải là một hoán vị của 1..m (mỗi số xuất hiện đúng một lần), ví dụ 1,1,2 là khóa sai.

Bảng 1.10. BAOMAT, khóa 2,3,1: đọc cột 2 "AA" + cột 3 "OT" + cột 1 "BM" = AAOTBM

	Cột 1	Cột 2	Cột 3
Hàng 1	B	A	O
Hàng 2	M	A	T

B. Tạo lớp TranspositionCipher

Bước 1.Nhấp phải package week_01 → New → Java Class... (nếu không thấy, chọn New → Other... → Java → Java Class).

Bước 2.Ô Class Name gõ TranspositionCipher → Finish. NetBeans mở tệp TranspositionCipher.java.

Bước 3.Bấm Ctrl+A, Delete để xóa nội dung mẫu, dán toàn bộ đoạn mã dưới đây, bấm Ctrl+S.

TranspositionCipher.java

```
package week_01;

/** Hoán vị cột theo khóa liệt kê thứ tự đọc các cột (đánh số từ 1). */
public final class TranspositionCipher {
    private TranspositionCipher() { }

    public static String encrypt(String text, int[] key) {
        return process(text, key, true);
    }

    public static String decrypt(String text, int[] key) {
        return process(text, key, false);
    }
```

```
    /** Đổi chuỗi khóa "2,3,1" (nhập trên form) thành mảng {2, 3, 1}. */
    public static int[] parseKey(String keyText) {
        String[] parts = keyText.trim().split("\\s*,\\s*");
        int[] key = new int[parts.length];
        try {
            for (int i = 0; i < parts.length; i++) {
                key[i] = Integer.parseInt(parts[i]);
            }
        } catch (NumberFormatException ex) {
            throw new IllegalArgumentException("Khóa phải là các số cách nhau bởi dấu phẩy, ví dụ 2,3,1.");
        }
        validateKey(key);
        return key;
    }

    private static void validateKey(int[] key) {
        if (key == null || key.length < 2 || key.length > 50) {
            throw new IllegalArgumentException("Khóa phải có từ 2 đến 50 cột.");
        }
        boolean[] used = new boolean[key.length + 1];
        for (int col : key) {
            if (col < 1 || col > key.length || used[col]) {
                throw new IllegalArgumentException("Khóa phải là hoán vị 1..m.");
            }
            used[col] = true;
        }
    }

    private static String process(String text, int[] key, boolean encrypt) {
        validateKey(key);
        int[] chars = text.codePoints().toArray();
        int length = chars.length;
        int columns = key.length;
        int rows = (length + columns - 1) / columns;
        int[][] grid = new int[rows][columns];
        int offset = 0;
        if (encrypt) {
            for (int i = 0; i < length; i++) {
                grid[i / columns][i % columns] = chars[i];
            }
            int[] result = new int[length];
            for (int column : key) {
                for (int row = 0; row < rows; row++) {
                    if (row * columns + column - 1 < length) {
                        result[offset++] = grid[row][column - 1];
                    }

                }
            }
            return new String(result, 0, result.length);
        }
        for (int column : key) {
            for (int row = 0; row < rows; row++) {
                if (row * columns + column - 1 < length) {
                    grid[row][column - 1] = chars[offset++];
                }
            }
        }
        int[] result = new int[length];
        for (int i = 0; i < length; i++) {
            result[i] = grid[i / columns][i % columns];
        }
        return new String(result, 0, result.length);
    }
}
```

GHI CHÚ

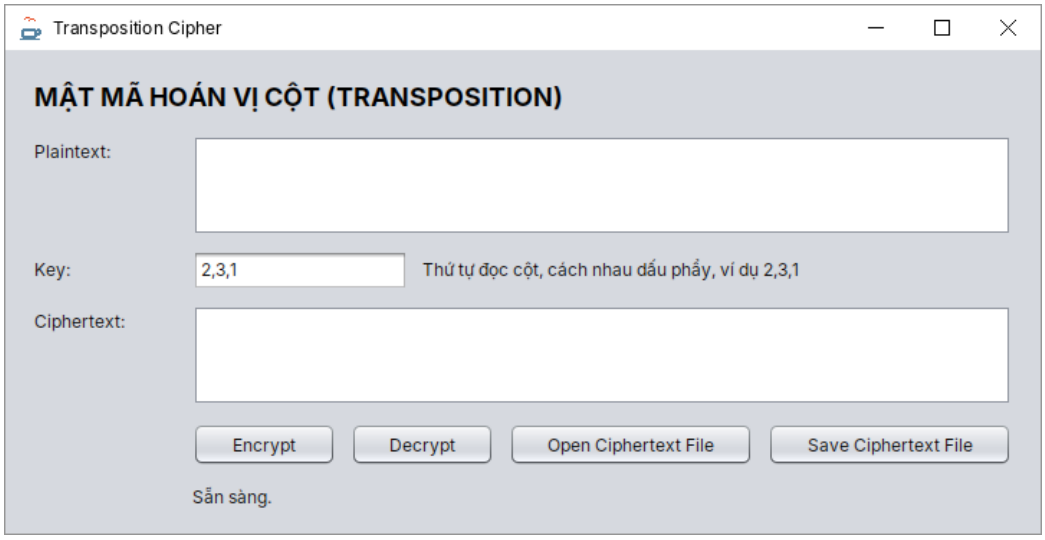
Hàm parseKey("2,3,1") đổi chuỗi khóa gõ trên form thành mảng số {2, 3, 1} và kiểm tra khóa có hợp lệ không.

C. Thiết kế form frm_Transposition

Bước 1.Trong Projects, nhấp phải frm_Caesar.java → Copy. Nhấp phải package week_01 → Paste → Refactor Copy... → ô New Name gõ frm_Transposition → Refactor. NetBeans tạo frm_Transposition.java (kèm tệp .form) và tự đổi tên lớp.

Bước 2.Mở frm_Transposition.java ở tab Design, sửa 4 chỗ trong bảng sau (các điều khiển và tên biến giữ nguyên).

Sửa ở đâu	Giá trị mới
Thuộc tính title của JFrame	Transposition Cipher
Label tiêu đề (jLabel1)	MẬT MÃ HOÁN VỊ CỘT (TRANSPOSITION)
Ô txt_key (thuộc tính text)	2,3,1
Label gợi ý cạnh ô Key (jLabel4)	Thứ tự đọc cột, cách nhau dấu phẩy, ví dụ 2,3,1



Hình 1.12. Form frm_Transposition sau khi sửa

D. Viết mã cho các nút

Bước 1.Nhấp đúp nút Encrypt (btn_encrypt) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_encryptActionPerformed

```
String plaintext = txt_plaintext.getText();
if (plaintext.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Hãy nhập Plaintext trước khi mã hóa.");
    return;
}
try {
    int[] key = TranspositionCipher.parseKey(txt_key.getText());
    txt_ciphertext.setText(TranspositionCipher.encrypt(plaintext, key));
    lbl_status.setText("Đã mã hóa với " + key.length + " cột.");
} catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(),
            "Dữ liệu không hợp lệ", JOptionPane.ERROR_MESSAGE);
}
```

Bước 2. Nhấp đúp nút Decrypt (btn_decrypt) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_decryptActionPerformed

```
String ciphertext = txt_ciphertext.getText();
if (ciphertext.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Hãy nhập Ciphertext trước khi giải mã.");
    return;
}
try {
    int[] key = TranspositionCipher.parseKey(txt_key.getText());
    txt_plaintext.setText(TranspositionCipher.decrypt(ciphertext, key));
    lbl_status.setText("Đã giải mã với " + key.length + " cột.");
} catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(),
            "Dữ liệu không hợp lệ", JOptionPane.ERROR_MESSAGE);
}
```

Bước 3. Nhấp đúp nút Save Ciphertext File (btn_save) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_saveActionPerformed.

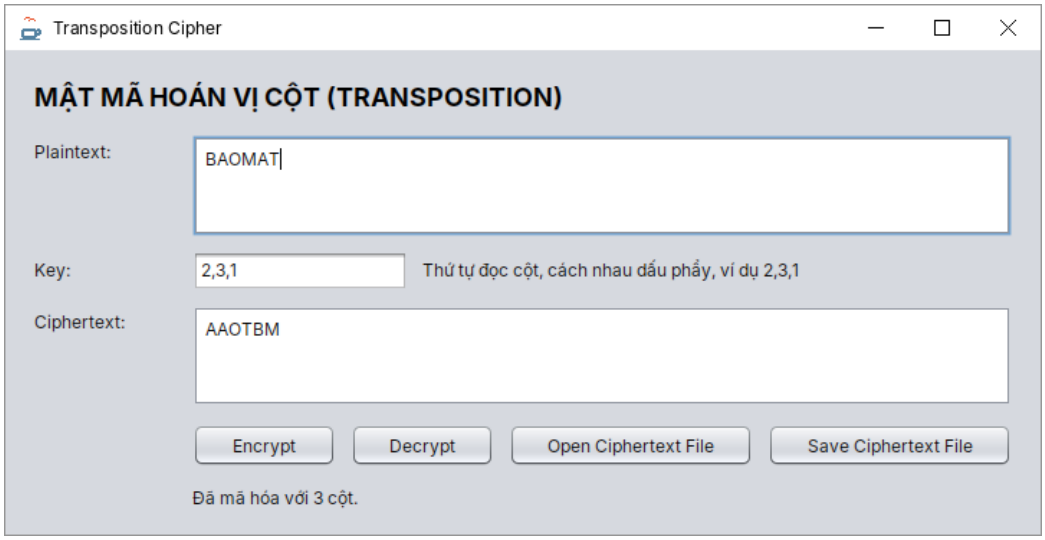
```
String path = FileTextIO.saveText(this, txt_ciphertext, "transposition_ciphertext.txt");
if (path != null) {
    lbl_status.setText("Đã lưu tệp: " + path);
}
```

Bước 4. Bấm Ctrl+S. Nếu NetBeans gạch đỏ dòng nào, rê chuột vào dòng đó để đọc lỗi và xem bảng lỗi ở cuối tài liệu.

E. Chạy và kiểm tra

Bước 1. Nhấp phải frm_Transposition.java → Run File (Shift+F6).

Bước 2. Nhập Plaintext BAOMAT, Key 2,3,1 → bấm Encrypt.



Hình 1.13. Kết quả mã hóa BAOMAT → AAOTBM

Bước 3. Xóa ô Plaintext, bấm Decrypt → ô Plaintext hiện BAOMAT.

Bước 4. Bấm Save Ciphertext File → lưu transposition_ciphertext.txt vào thư mục bất kỳ. Xóa ô Ciphertext, bấm Open Ciphertext File → chọn lại tệp vừa lưu → bản mã hiện lại.

Bước 5. Thử khóa sai: Key = 1,1,2 → bấm Encrypt → phải hiện hộp thông báo lỗi "Khóa phải là hoán vị 1..m.", chương trình không bị tắt.

BÀI 1.6. MẬT MÃ AUTOKEY

A. Hiểu thuật toán

- Giống Vigenère nhưng khóa **không lặp lại**: dòng khóa = khóa mồi + **chính bản rõ** (KEY + HELLO...).
- Mã hóa: $\mathbf{C_i = (P_i + K_i) \bmod 26}$. Giải mã: tìm được chữ bản rõ nào thì nối ngay chữ đó vào dòng khóa để giải các chữ tiếp theo.

Bảng 1.11. HELLO + khóa mồi KEY → RIJSS

Bản rõ	H (7)	E (4)	L (11)	L (11)	O (14)
Dòng khóa	K (10)	E (4)	Y (24)	H (7)	E (4)
Tổng mod 26	17	8	9	18	18
Bản mã	R	I	J	S	S

B. Tạo lớp AutokeyCipher

Bước 1. Nhấp phải package week_01 → New → Java Class... (nếu không thấy, chọn New → Other... → Java → Java Class).

Bước 2. Ô Class Name gõ AutokeyCipher → Finish. NetBeans mở tệp AutokeyCipher.java.

Bước 3. Bấm Ctrl+A, Delete để xóa nội dung mẫu, dán toàn bộ đoạn mã dưới đây, bấm Ctrl+S.

AutokeyCipher.java

```
package week_01;

import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

/** Plaintext Autokey: sau khóa khởi tạo, bổ sung các chữ của bản rõ. */
public final class AutokeyCipher {
    private AutokeyCipher() { }

    public static String encrypt(String text, String key) {
        return process(text, key, true);
    }

    public static String decrypt(String text, String key) {
        return process(text, key, false);
    }

    private static String process(String text, String key, boolean encrypt) {
        String k = key.trim().toUpperCase(Locale.ROOT);
        if (!k.matches("[A-Z]+")) {
            throw new IllegalArgumentException("Khóa Autokey chỉ gồm A-Z.");
        }
        List<Integer> plainOffsets = new ArrayList<>();
        StringBuilder result = new StringBuilder();
        int letterIndex = 0;
        for (char c : text.toCharArray()) {
            char base;
            if (c >= 'A' && c <= 'Z') base = 'A';
            else if (c >= 'a' && c <= 'z') base = 'a';
            else {
                result.append(c);
                continue;
            }
            int shift;
```

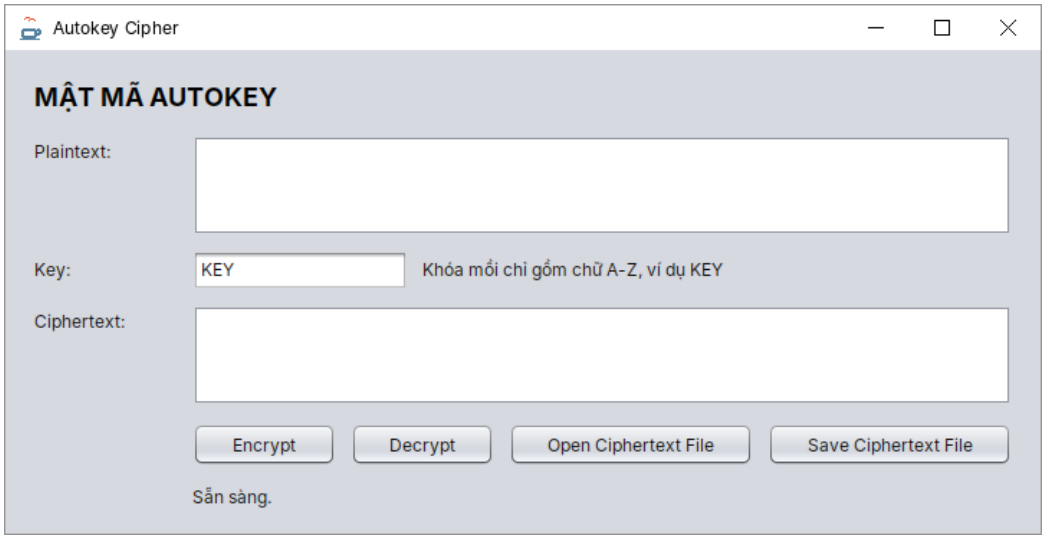
```
            if (letterIndex < k.length()) {
                shift = k.charAt(letterIndex) - 'A';
            } else {
                shift = plainOffsets.get(letterIndex - k.length());
            }
            int delta = encrypt ? shift : -shift;
            char transformed = (char) (base + Math.floorMod(c - base + delta, 26));
            result.append(transformed);
            // Giải mã phải thêm chữ bản rõ vừa KHÔI PHỤC, không thêm bản mã.
            char plaintextChar = encrypt ? c : transformed;
            plainOffsets.add(plaintextChar - base);
            letterIndex++;
        }
        return result.toString();
    }
}
```

C. Thiết kế form frm_Autokey

Bước 1. Trong Projects, nhấp phải frm_Caesar.java → Copy. Nhấp phải package week_01 → Paste → Refactor Copy... → ô New Name gõ frm_Autokey → Refactor. NetBeans tạo frm_Autokey.java (kèm tệp .form) và tự đổi tên lớp.

Bước 2. Mở frm_Autokey.java ở tab Design, sửa 4 chỗ trong bảng sau (các điều khiển và tên biến giữ nguyên).

Sửa ở đâu	Giá trị mới
Thuộc tính title của JFrame	Autokey Cipher
Label tiêu đề (jLabel1)	MẬT MÃ AUTOKEY
Ô txt_key (thuộc tính text)	KEY
Label gợi ý cạnh ô Key (jLabel4)	Khóa mỗi chỉ gồm chữ A-Z, ví dụ KEY



Hình 1.14. Form frm_Autokey sau khi sửa

D. Viết mã cho các nút

Bước 1.Nhấp đúp nút Encrypt (btn_encrypt) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_encryptActionPerformed

```
String plaintext = txt_plaintext.getText();
if (plaintext.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Hãy nhập Plaintext trước khi mã hóa.");
    return;
}
try {
    String key = txt_key.getText().trim();
    txt_ciphertext.setText(AutokeyCipher.encrypt(plaintext, key));
    lbl_status.setText("Đã mã hóa Autokey với khóa mồi " + key + ".");
} catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(),
            "Dữ liệu không hợp lệ", JOptionPane.ERROR_MESSAGE);
}
```

Bước 2. Nhấp đúp nút Decrypt (btn_decrypt) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_decryptActionPerformed

```
String ciphertext = txt_ciphertext.getText();
if (ciphertext.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Hãy nhập Ciphertext trước khi giải mã.");
    return;
}
try {
    String key = txt_key.getText().trim();
    txt_plaintext.setText(AutokeyCipher.decrypt(ciphertext, key));
    lbl_status.setText("Đã giải mã Autokey với khóa mồi " + key + ".");
} catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(),
            "Dữ liệu không hợp lệ", JOptionPane.ERROR_MESSAGE);
}
```

Bước 3. Nhấp đúp nút Save Ciphertext File (btn_save) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_saveActionPerformed

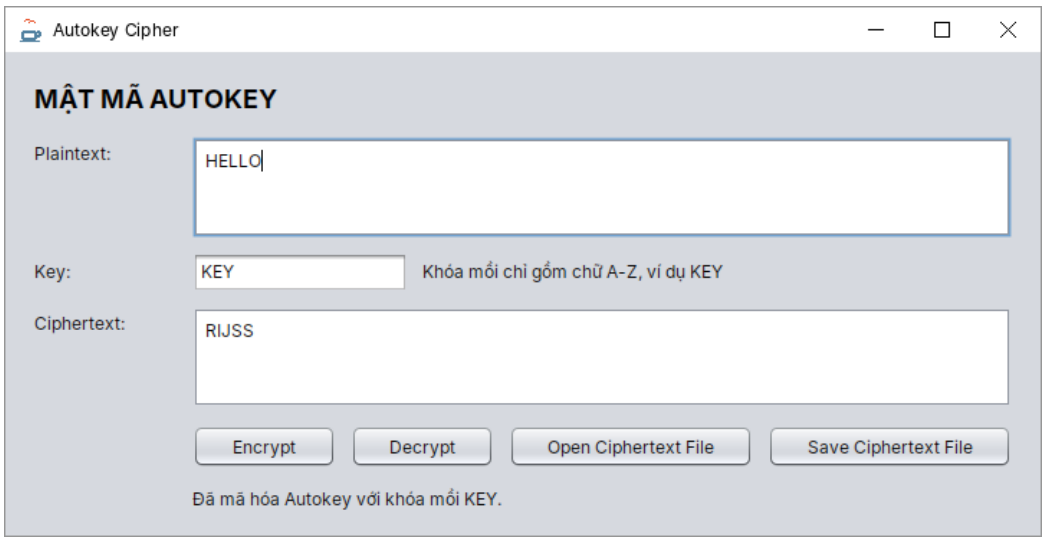
```
String path = FileTextIO.saveText(this, txt_ciphertext, "autokey_ciphertext.txt");
if (path != null) {
    lbl_status.setText("Đã lưu tệp: " + path);
}
```

Bước 4.Bấm Ctrl+S. Nếu NetBeans gạch đỏ dòng nào, rê chuột vào dòng đó để đọc lỗi và xem bảng lỗi ở cuối tài liệu.

E. Chạy và kiểm tra

Bước 1. Nhấp phải frm_Autokey.java → Run File (Shift+F6).

Bước 2. Nhập Plaintext HELLO, Key KEY → bấm Encrypt.



Hình 1.15. Kết quả mã hóa HELLO → RIJSS

Bước 3. Xóa ô Plaintext, bấm Decrypt → ô Plaintext hiện HELLO.

Bước 4. Bấm Save Ciphertext File → lưu autokey_ciphertext.txt vào thư mục bất kỳ. Xóa ô Ciphertext, bấm Open Ciphertext File → chọn lại tệp vừa lưu → bản mã hiện lại.

Bước 5. Thử khóa sai: Key = 123 → bấm Encrypt → phải hiện hộp thông báo lỗi "Khóa Autokey chỉ gồm A-Z.", chương trình không bị tắt.

PHẦN 3. BÀI NÂNG CAO 01 – CHƯƠNG TRÌNH TỔNG HỢP VÀ LƯU LỊCH SỬ SQL SERVER

Mục tiêu: một form duy nhất chọn được 6 thuật toán bằng JComboBox; mỗi lần Encrypt/Decrypt ghi một dòng vào bảng dbo.history của SQL Server, xem lại trên JTable và trong SSMS. Làm theo đúng thứ tự: 3.1 lớp tổng hợp → 3.2 lớp kết nối DbConfig/HistoryDao → 3.3 form tổng hợp (chạy thử được khi chưa có SQL) → 3.4 cài SQL Server → 3.5 form kết nối/lịch sử rồi nối nốt hai nút → 3.6 kiểm tra.

3.1. Tạo lớp CipherTongHop

Bước 1. Nhấp phải package week_01 → New → Java Class... (nếu không thấy, chọn New → Other... → Java → Java Class).

Bước 2. Ô Class Name gõ CipherTongHop → Finish. NetBeans mở tệp CipherTongHop.java.

Bước 3. Bấm Ctrl+A, Delete để xóa nội dung mẫu, dán toàn bộ đoạn mã dưới đây, bấm Ctrl+S.

CipherTongHop.java

```
package week_01;

/**
 * Bài nâng cao 01: gọi đúng lớp mã hóa theo tên thuật toán chọn trên JComboBox.
 * Khóa luôn nhập dạng chuỗi; lớp này tự đổi sang kiểu số khi thuật toán cần.
 */
public final class CipherTongHop {

    /** Tên hiển thị trên JComboBox - phải giống hệt model của cbo_algorithm. */
    public static final String[] ALGORITHMS = {"Caesar", "Vigenere", "Rail Fence", "Playfair", "Transposition", "Autokey"};

    private CipherTongHop() {
    }

    public static String encrypt(String algorithm, String text, String key) {
        return run(algorithm, text, key.trim(), true);
    }

    public static String decrypt(String algorithm, String text, String key) {
        return run(algorithm, text, key.trim(), false);
    }

    /** Gợi ý khóa mẫu cho từng thuật toán (hiện cạnh JComboBox). */
    public static String sampleKey(String algorithm) {
        switch (algorithm) {
            case "Caesar": return "3";
            case "Rail Fence": return "3";
            case "Playfair": return "KEYWORD";
            case "Transposition": return "2,3,1";
            default: return "KEY";
        }
    }

    private static String run(String algorithm, String text, String key, boolean encrypt) {
        if (text.isEmpty()) {
            throw new IllegalArgumentException("Chưa nhập dữ liệu.");
        }
        switch (algorithm) {
            case "Caesar":
```

```
                int k = parseInt(key, "Khóa Caesar phải là số nguyên, ví dụ 3.");
                return encrypt ? CaesarCipher.encrypt(text, k) : CaesarCipher.decrypt(text, k);
            case "Vigenere":
                return encrypt ? VigenereCipher.encrypt(text, key) : VigenereCipher.decrypt(text, key);
            case "Rail Fence":
                int rails = parseInt(key, "Số hàng Rail Fence phải là số nguyên, ví dụ 3.");
                return encrypt ? RailFenceCipher.encrypt(text, rails) : RailFenceCipher.decrypt(text, rails);
            case "Playfair":
                return encrypt ? PlayfairCipher.encrypt(text, key) : PlayfairCipher.decrypt(text, key);
            case "Transposition":
                int[] order = TranspositionCipher.parseKey(key);
                return encrypt ? TranspositionCipher.encrypt(text, order) : TranspositionCipher.decrypt(text, order);
            case "Autokey":
                return encrypt ? AutokeyCipher.encrypt(text, key) : AutokeyCipher.decrypt(text, key);
            default:
                throw new IllegalArgumentException("Thuật toán không hỗ trợ: " + algorithm);
        }
    }

    private static int parseInt(String key, String message) {
        try {
            return Integer.parseInt(key);
        } catch (NumberFormatException ex) {
            throw new IllegalArgumentException(message);
        }
    }
}
```

- Lệnh switch (algorithm) chọn đúng lớp thuật toán theo chữ đang chọn trên JComboBox, vì vậy danh sách của Combo Box phải gõ giống hệt mảng ALGORITHMS.
- sampleKey(...) trả về khóa mẫu để tự điền vào ô Key khi đổi thuật toán.

3.2. Tạo lớp kết nối DbConfig và HistoryDao

Bước 1. Tạo lớp DbConfig (thao tác: nhấp phải week_01 → New → Java Class...). Lớp này giữ thông tin kết nối mà bạn nhập trên form.

DbConfig.java

```
package week_01;

import java.sql.Connection;
import java.sql.DatabaseMetaData;
import java.sql.DriverManager;
import java.sql.SQLException;

/**
 * Lưu thông tin kết nối SQL Server mà sinh viên nhập trên form frm_KetNoiSQL.
 * Mật khẩu chỉ nằm trong bộ nhớ khi chương trình chạy, KHÔNG ghi vào mã nguồn hay tệp.
 */
public final class DbConfig {

    private static String server;
    private static int port;
    private static String database;
    private static String user;
    private static char[] password;

    private DbConfig() {
    }

    /** Ghi nhớ thông tin kết nối (gọi khi bấm nút Test Connection). */
    public static synchronized void set(String newServer, int newPort, String newDatabase,
            String newUser, char[] newPassword) {
        if (newServer.isBlank() || newDatabase.isBlank() || newUser.isBlank()) {
            throw new IllegalArgumentException("Server, Database và User không được để trống.");
        }
        if (newPort < 1 || newPort > 65535) {
            throw new IllegalArgumentException("Port phải nằm trong khoảng 1 - 65535.");
```

```
        }
        server = newServer;
        port = newPort;
        database = newDatabase;
        user = newUser;
        password = newPassword.clone();
    }

    /** Xóa cấu hình (dùng khi kết nối thất bại). */
    public static synchronized void clear() {
        server = null;
        password = null;
    }

    /** true nếu đã nhập cấu hình và kết nối thử thành công. */
    public static synchronized boolean isReady() {
        return server != null;
    }

    /** Mở một kết nối JDBC mới; nhớ đóng bằng try-with-resources. */
    public static synchronized Connection open() throws SQLException {
        if (server == null) {
            throw new SQLException("Chưa cấu hình SQL Server. Hãy mở form Kết nối SQL Server và bấm Test Connection.");
        }
        String url = "jdbc:sqlserver://" + server + ":" + port
                + ";databaseName=" + database
                + ";encrypt=true;trustServerCertificate=true;loginTimeout=5";
        return DriverManager.getConnection(url, user, new String(password));
    }

    /** Kết nối thử; trả về chuỗi mô tả để hiện lên form. */
    public static String test() throws SQLException {
        try (Connection con = open()) {
            DatabaseMetaData md = con.getMetaData();
            return "Kết nối thành công: " + md.getDatabaseProductName() + " "
                    + md.getDatabaseProductVersion() + " - database " + con.getCatalog();
        }
    }
}
```

- Chuỗi kết nối: jdbc:sqlserver://127.0.0.1:14333;databaseName=...;encrypt=true;trustServerCertificate=true - trustServerCertificate=true để chấp nhận chứng chỉ tự ký của SQL Server Express trên máy thực hành.
- Mật khẩu kiểu char[] lấy từ ô **Password Field**, chỉ tồn tại khi chương trình chạy.

Bước 2. Tạo lớp HistoryDao - các câu lệnh SQL dùng PreparedStatement (dấu ?) để tránh lỗi SQL Injection.

HistoryDao.java

```
package week_01;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import javax.swing.table.DefaultTableModel;

/** Ghi và đọc bảng dbo.history trong database LAB1_MaHoaCoDien. */
public final class HistoryDao {

    private HistoryDao() {
    }

    /** Thêm 1 dòng lịch sử. operation chỉ nhận ENCRYPT hoặc DECRYPT (ràng buộc CHECK trong SQL). */
```

```
    public static void insert(String algorithm, String operation, String input, String result) throws SQLException {
        String sql = "INSERT INTO dbo.history(algorithm, operation, input_text, result_text) VALUES (?, ?, ?, ?)";
        try (Connection con = DbConfig.open(); PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, algorithm);
            ps.setString(2, operation);
            ps.setString(3, input);
            ps.setString(4, result);
            ps.executeUpdate();
        }
    }

    /** Thử ghi lịch sử, không làm dừng chương trình; trả về câu thông báo để hiện trên form. */
    public static String tryInsert(String algorithm, String operation, String input, String result) {
        if (!DbConfig.isReady()) {
            return "Chưa kết nối SQL Server nên chưa lưu lịch sử.";
        }
        try {
            insert(algorithm, operation, input, result);
            return "Đã lưu lịch sử vào SQL Server.";
        } catch (SQLException ex) {
            return "Lỗi lưu lịch sử: " + ex.getMessage();
        }
    }

    /** Bảng rỗng có sẵn tên cột (dùng khi mở form). */
    public static DefaultTableModel emptyModel() {
        return new DefaultTableModel(new Object[] {"ID", "Thời gian", "Thuật toán", "Thao tác", "Đầu vào", "Kết quả"}, 0) {
            @Override
            public boolean isCellEditable(int row, int column) {
                return false;
            }
        };
    }

    /** Đọc tối đa 'top' dòng mới nhất, trả về model để gán cho JTable. */
    public static DefaultTableModel loadLatest(int top) throws SQLException {
        DefaultTableModel model = emptyModel();
        String sql = "SELECT TOP (?) id, created_at, algorithm, operation, input_text, result_text "
                + "FROM dbo.history ORDER BY id DESC";
        try (Connection con = DbConfig.open(); PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, top);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    model.addRow(new Object[] {rs.getLong(1), rs.getTimestamp(2), rs.getString(3),
                        rs.getString(4), rs.getString(5), rs.getString(6)});
                }
            }
        }
        return model;
    }
}
```

- tryInsert(...): nếu **chưa kết nối** thì bỏ qua và trả về câu thông báo, nên frm_TongHop vẫn chạy được khi không có SQL Server.
- loadLatest(50): đọc 50 dòng mới nhất, trả về DefaultTableModel để gán thẳng cho JTable.

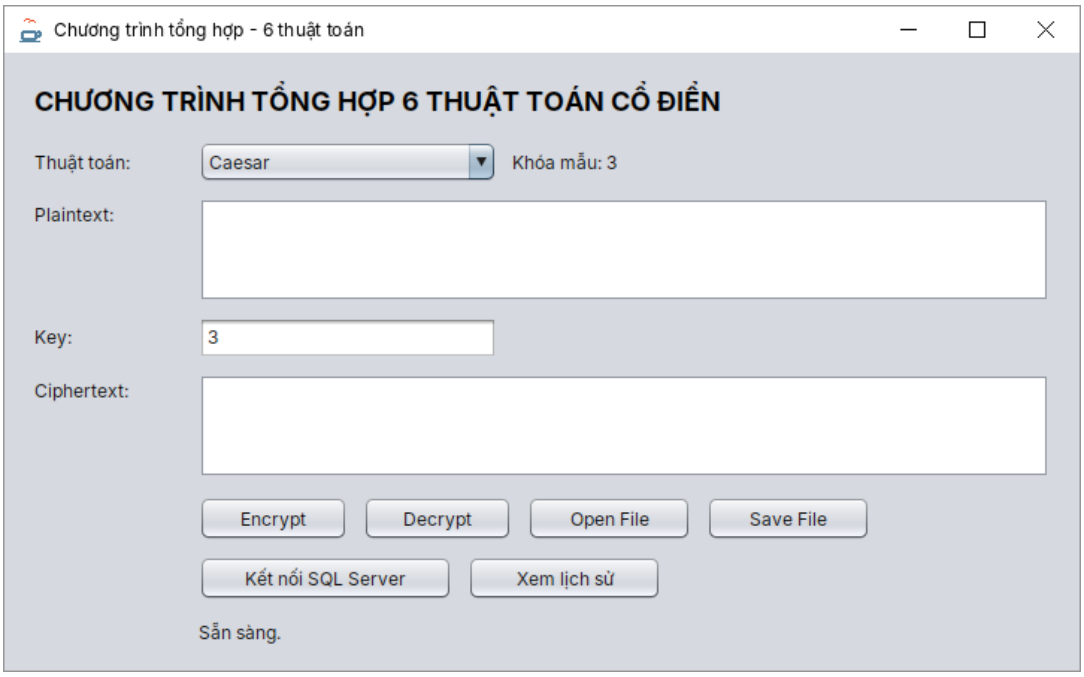
3.3. Thiết kế form frm_TongHop

Bước 1. Tạo JFrame Form tên frm_TongHop (thao tác A). Đặt **title** = Chương trình tổng hợp - 6 thuật toán, **defaultCloseOperation** = DISPOSE.

Bước 2. Đặt các điều khiển theo **thứ tự trong bảng** (từ trên xuống, trái sang phải), đổi tên biến và đặt thuộc tính đúng như bảng.

Bảng 1.12. Điều khiển trên form frm_TongHop

#	Palette (Swing Controls)	Variable Name (đổi tên)	Thuộc tính cần đặt (Properties)	Cỡ gợi ý (R×C px)
1	Label	(giữ tên mặc định)	text = CHƯƠNG TRÌNH TỔNG HỢP 6 THUẬT TOÁN CỔ ĐIỂN font = Segoe UI 18 Bold	570×34
2	Label	(giữ tên mặc định)	text = Thuật toán:	100×22
3	Combo Box	cbo_algorithm	model = Caesar, Vigenere, Rail Fence, Playfair, Transposition, Autokey	200×28
4	Label	lbl_hint	text = Khóa mẫu: 3	360×22
5	Label	(giữ tên mặc định)	text = Plaintext:	100×22
6	Text Area	txt_plaintext	lineWrap = tích rows = 3	570×70
7	Label	(giữ tên mặc định)	text = Key:	100×22
8	Text Field	txt_key	text = 3	200×28
9	Label	(giữ tên mặc định)	text = Ciphertext:	100×22
10	Text Area	txt_ciphertext	lineWrap = tích rows = 3	570×70
11	Button	btn_encrypt	text = Encrypt	100×30
12	Button	btn_decrypt	text = Decrypt	100×30
13	Button	btn_open	text = Open File	110×30
14	Button	btn_save	text = Save File	110×30
15	Button	btn_sql	text = Kết nối SQL Server	170×30
16	Button	btn_history	text = Xem lịch sử	130×30
17	Label	lbl_status	text = Sẵn sàng.	570×22



Hình 1.16. Form frm_TongHop sau khi thiết kế (chạy thử bằng Shift+F6)

Bước 3. Bấm **Ctrl+S**, rồi nhấp phải tệp → **Run File** để xem thử: cửa sổ phải giống Hình 1.16 (bấm nút lúc này chưa có tác dụng).

Viết mã cho form frm_TongHop

Bước 1. Thêm import javax.swing.JOptionPane; và setLocationRelativeTo(null); (như mục 1.2).

Bước 2. Nhấp phải ô Combo Box → Events → Action → actionPerformed (hoặc nhấp đúp ô) (cbo_algorithm) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm cbo_algorithmActionPerformed

```
String algorithm = (String) cbo_algorithm.getSelectedItem();
txt_key.setText(CipherTongHop.sampleKey(algorithm));
lbl_hint.setText("Khóa mẫu: " + CipherTongHop.sampleKey(algorithm));
```

Bước 3. Nhấp đúp nút Encrypt (btn_encrypt) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_encryptActionPerformed

```
String algorithm = (String) cbo_algorithm.getSelectedItem();
String input = txt_plaintext.getText();
try {
    String result = CipherTongHop.encrypt(algorithm, input, txt_key.getText());
    txt_ciphertext.setText(result);
    String saved = HistoryDao.tryInsert(algorithm, "ENCRYPT", input, result);
    lbl_status.setText("Đã mã hóa " + algorithm + ". " + saved);
} catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(),
            "Dữ liệu không hợp lệ", JOptionPane.ERROR_MESSAGE);
}
```

Bước 4. Nhấp đúp nút Decrypt (btn_decrypt) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_decryptActionPerformed

```
String algorithm = (String) cbo_algorithm.getSelectedItem();
String input = txt_ciphertext.getText();
try {
    String result = CipherTongHop.decrypt(algorithm, input, txt_key.getText());
    txt_plaintext.setText(result);
    String saved = HistoryDao.tryInsert(algorithm, "DECRYPT", input, result);
    lbl_status.setText("Đã giải mã " + algorithm + ". " + saved);
} catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(),
            "Dữ liệu không hợp lệ", JOptionPane.ERROR_MESSAGE);
}
```

Bước 5. Nhấp đúp nút Open File (btn_open) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_openActionPerformed

```
String path = FileTextIO.openText(this, txt_ciphertext);
if (path != null) {
    lbl_status.setText("Đã mở tệp: " + path);
}
```

Bước 6. Nhấp đúp nút Save File (btn_save) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_saveActionPerformed

```
String path = FileTextIO.saveText(this, txt_ciphertext, "ciphertext.txt");
if (path != null) {
    lbl_status.setText("Đã lưu tệp: " + path);
}
```

Bước 7. Bấm Ctrl+S. Nếu NetBeans gạch đỏ dòng nào, rê chuột vào dòng đó để đọc lỗi và xem bảng lỗi ở cuối tài liệu. Lúc này project phải biên dịch được (Shift+F11 →

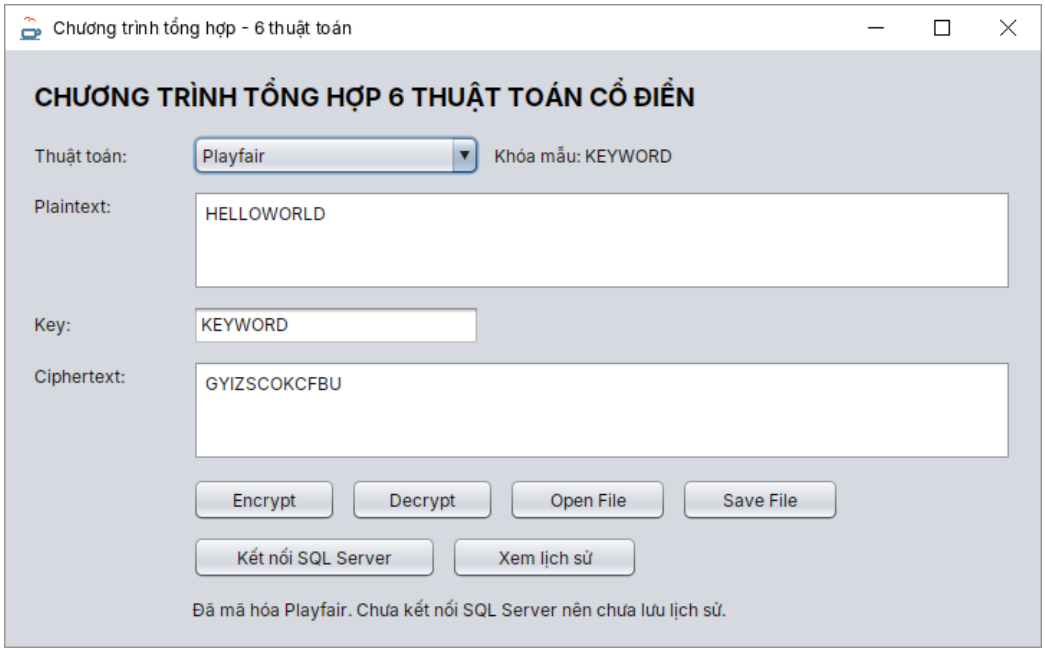
BUILD SUCCESS). Hai nút Kết nối SQL Server và Xem lịch sử sẽ được nối ở cuối mục 3.5, nên bấm vào chúng bây giờ chưa có tác dụng.

> **GHI CHÚ**
>
> Form tổng hợp chạy được ngay cả khi chưa cài SQL Server: khi chưa kết nối, dòng trạng thái chỉ báo "Chưa kết nối SQL Server nên chưa lưu lịch sử". Bạn có thể kiểm thử 6 thuật toán ngay bây giờ rồi mới làm tiếp phần SQL.

Kiểm thử đủ 6 thuật toán

Chạy frm_TongHop (Shift+F6). Lần lượt chọn từng thuật toán (ô Key tự điền khóa mẫu), nhập Plaintext, bấm **Encrypt**:

Chọn	Plaintext	Key (tự điền)	Ciphertext đúng
Caesar	HELLO	3	KHOOR
Vigenere	HELLO	KEY	RIJVS
Rail Fence	HELLO WORLD	3	HOREL OLLWD
Playfair	HELLOWORLD	KEYWORD	GYIZSCOKCFBU
Transposition	BAOMAT	2,3,1	AAOTBM
Autokey	HELLO	KEY	RIJSS



Hình 1.17. frm_TongHop với Playfair; dòng trạng thái cho biết chưa kết nối SQL nên chưa lưu lịch sử

3.4. Cài đặt và cấu hình Microsoft SQL Server

Phần này chỉ cần làm **một lần trên mỗi máy**; các Lab 2–5 dùng lại đúng cấu hình này (cùng instance SQLEXPRESS, cùng cổng TCP 14333).

a) Cài SQL Server Express và SSMS

Bước 1. Tải SQL Server 2022 Express tại trang Microsoft (tìm "SQL Server 2022 Express download"), chạy tệp cài đặt → chọn Basic → Accept → Install. Ghi nhớ tên instance hiển thị cuối quá trình cài: SQLEXPRESS.

Bước 2. Tải và cài SQL Server Management Studio (SSMS) bản mới nhất (tìm "Download SSMS").

b) Bật giao thức TCP/IP và đặt cổng 14333

Bước 1. Mở SQL Server 2022 Configuration Manager (gõ SQLServerManager16.msc vào ô tìm kiếm của Windows; bản 2019 là SQLServerManager15.msc).

Bước 2. Bên trái chọn SQL Server Network Configuration → Protocols for SQLEXPRESS. Bên phải nhấp phải TCP/IP → Enable.

Bước 3. Nhấp đúp TCP/IP → tab IP Addresses → kéo xuống cuối, mục IPAll: xóa trống ô TCP Dynamic Ports, ô TCP Port gõ 14333 → OK.

Bước 4. Bên trái chọn SQL Server Services → nhấp phải SQL Server (SQLEXPRESS) → Restart.

c) Bật chế độ đăng nhập SQL Server (Mixed Mode)

Bước 1. Mở SSMS. Hộp Connect to Server: Server type = Database Engine; Server name = localhost\SQLEXPRESS; Authentication = Windows Authentication; tích Trust server certificate → Connect.

Bước 2. Trong Object Explorer, nhấp phải tên server (dòng trên cùng) → Properties → Security → chọn SQL Server and Windows Authentication mode → OK.

Bước 3. Nhấp phải tên server → Restart → Yes (bắt buộc để chế độ mới có hiệu lực).

> ➢ **KẾT QUẢ ĐÚNG PHẢI THẤY**
>
> Configuration Manager: TCP/IP = **Enabled**, IPAll → TCP Port = 14333; dịch vụ SQL Server (SQLEXPRESS) = **Running**.

Chạy các script tạo database và tài khoản

Bước 1. Trong SSMS (đang kết nối bằng Windows Authentication), chọn File → Open → File... (Ctrl+O) → mở 03_SQL_SERVER\01_Tao_CSDL.sql → bấm Execute (F5).

01_Tao_CSDL.sql

```
-- Chạy bằng SSMS khi đăng nhập quyền sysadmin / dbcreator.
-- Được phép chạy lại: không xóa dữ liệu hiện có.
IF DB_ID(N'LAB1_MaHoaCoDien') IS NULL
    CREATE DATABASE [LAB1_MaHoaCoDien];
GO
USE [LAB1_MaHoaCoDien];
GO
IF OBJECT_ID(N'dbo.history', N'U') IS NULL
BEGIN
    CREATE TABLE dbo.history (
        id BIGINT IDENTITY(1,1) NOT NULL PRIMARY KEY,
        created_at DATETIME2(0) NOT NULL DEFAULT SYSDATETIME(),
        algorithm NVARCHAR(50) NOT NULL,
        operation VARCHAR(10) NOT NULL,
        input_text NVARCHAR(MAX) NOT NULL,
        result_text NVARCHAR(MAX) NOT NULL,
        CONSTRAINT CK_lab1_operation CHECK (operation IN ('ENCRYPT','DECRYPT'))
    );
END;
GO
SELECT DB_NAME() AS ten_co_so_du_lieu, COUNT(*) AS so_ban_ghi FROM dbo.history;
GO
```

Bước 2. Mở 02_Tao_Login_MAU.sql. KHÔNG cần sửa gì: script đã đặt sẵn mật khẩu LabUser@2026 cho tài khoản của bài (chỉ dùng trên máy cá nhân). Bấm Execute (F5). Ghi nhớ mật khẩu này: bạn sẽ gõ nó vào form Kết nối SQL Server.

02_Tao_Login_MAU.sql

```
-- CHỈ DÀNH CHO LAB TRÊN MÁY CÁ NHÂN. Chạy với quyền quản trị SQL Server.
-- Mật khẩu lab: LabUser@2026 - chỉ dùng trên máy cá nhân; nếu đổi thì gõ đúng mật khẩu mới trên form kết nối.
USE [master];
GO
IF NOT EXISTS (SELECT 1 FROM sys.sql_logins WHERE name = N'lab1_user')
BEGIN
    CREATE LOGIN [lab1_user]
      WITH PASSWORD = 'LabUser@2026',
           CHECK_POLICY = ON,
           CHECK_EXPIRATION = OFF;
END;
GO
USE [LAB1_MaHoaCoDien];
GO
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = N'lab1_user')
    CREATE USER [lab1_user] FOR LOGIN [lab1_user];
GO
-- Cấp quyền vừa đủ: SELECT, INSERT, DELETE lịch sử. Không cấp db_owner.
GRANT SELECT, INSERT, DELETE ON OBJECT::dbo.history TO [lab1_user];
GO
```

Bước 3. Kiểm tra đăng nhập bằng tài khoản mới: trong Object Explorer bấm Connect → Database Engine... → Server name = 127.0.0.1,14333 (dấu phẩy trước số cổng) → Authentication = SQL Server Authentication → Login = lab1_user, Password = LabUser@2026 → tích Trust server certificate → Connect.

KẾT QUẢ ĐÚNG PHẢI THẤY

Kết nối thứ hai xuất hiện trong Object Explorer với tên 127.0.0.1,14333 (... - lab1_user); mở **Databases** thấy LAB1_MaHoaCoDien.

Kết quả của script 01 hiển thị ten_co_so_du_lieu = LAB1_MaHoaCoDien, so_ban_ghi = 0.

Đăng nhập được ở bước này thì chương trình Java chắc chắn kết nối được (cùng địa chỉ, cổng, tài khoản).

3.5. Tạo form frm_KetNoiSQL và frm_LichSu

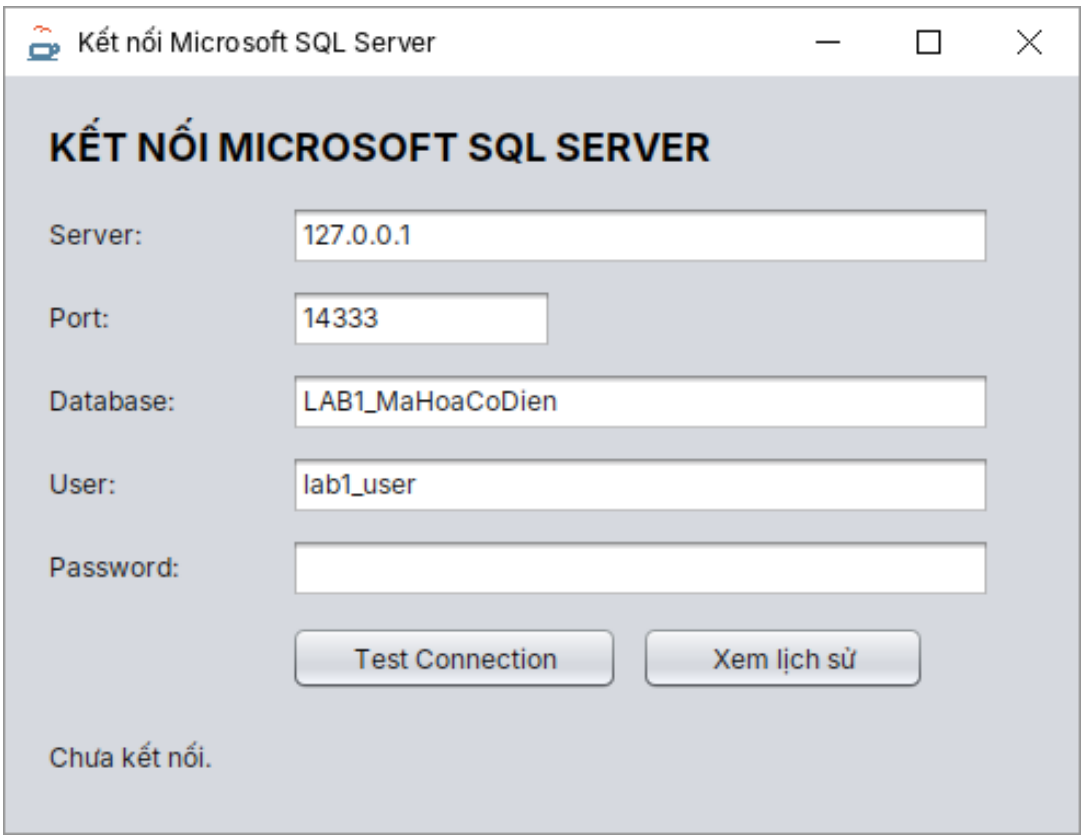
Form frm_KetNoiSQL

Bước 1. Tạo JFrame Form tên frm_KetNoiSQL (thao tác A). Đặt title = Kết nối Microsoft SQL Server, defaultCloseOperation = DISPOSE.

Bước 2. Đặt các điều khiển theo thứ tự trong bảng (từ trên xuống, trái sang phải), đổi tên biến và đặt thuộc tính đúng như bảng.

Bảng 1.13. Điều khiển trên form frm_KetNoiSQL

#	Palette (Swing Controls)	Variable Name (đổi tên)	Thuộc tính cần đặt (Properties)	Cỡ gợi ý (R×C px)
1	Label	(giữ tên mặc định)	text = KẾT NỐI MICROSOFT SQL SERVER font = Segoe UI 18 Bold	450×34
2	Label	(giữ tên mặc định)	text = Server:	100×22
3	Text Field	txt_server	text = 127.0.0.1	320×28
4	Label	(giữ tên mặc định)	text = Port:	100×22
5	Text Field	txt_port	text = 14333	120×28
6	Label	(giữ tên mặc định)	text = Database:	100×22
7	Text Field	txt_database	text = LAB1_MaHoaCoDien	320×28
8	Label	(giữ tên mặc định)	text = User:	100×22
9	Text Field	txt_user	text = lab1_user	320×28
10	Label	(giữ tên mặc định)	text = Password:	100×22
11	Password Field	txt_password	text = (xóa trống)	320×28
12	Button	btn_test	text = Test Connection	150×30
13	Button	btn_history	text = Xem lịch sử	130×30
14	Label	lbl_status	text = Chưa kết nối.	450×40



Hình 1.18. Form frm_KetNoiSQL sau khi thiết kế (chạy thử bằng Shift+F6)

Bước 3. Bấm Ctrl+S, rồi nhấp phải tệp → Run File để xem thử: cửa sổ phải giống Hình 1.18 (bấm nút lúc này chưa có tác dụng).

Bước 4. Thêm import javax.swing.JOptionPane; và setLocationRelativeTo(null);.

Bước 5. Nhấp đúp nút Test Connection (btn_test) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_testActionPerformed

```
try {
    int port = Integer.parseInt(txt_port.getText().trim());
    DbConfig.set(txt_server.getText().trim(), port, txt_database.getText().trim(),
            txt_user.getText().trim(), txt_password.getPassword());
    lbl_status.setText("<html>" + DbConfig.test() + "</html>");
} catch (Exception ex) {
    DbConfig.clear();
    lbl_status.setText("<html>Kết nối thất bại: " + ex.getMessage() + "</html>");
    JOptionPane.showMessageDialog(this, ex.getMessage(),
            "Không kết nối được SQL Server", JOptionPane.ERROR_MESSAGE);
}
```

Bước 6. Bấm Ctrl+S. Nếu NetBeans gạch đỏ dòng nào, rê chuột vào dòng đó để đọc lỗi và xem bảng lỗi ở cuối tài liệu.

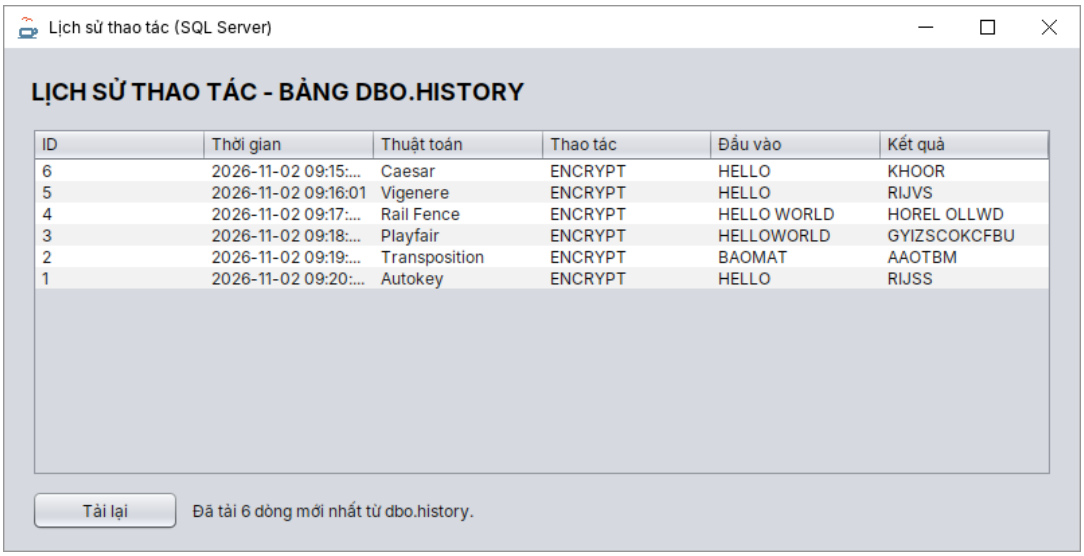
Form frm_LichSu

Bước 1. Tạo JFrame Form tên frm_LichSu (thao tác A). Đặt title = Lịch sử thao tác (SQL Server), defaultCloseOperation = DISPOSE.

Bước 2. Đặt các điều khiển theo thứ tự trong bảng (từ trên xuống, trái sang phải), đổi tên biến và đặt thuộc tính đúng như bảng.

Bảng 1.14. Điều khiển trên form frm_LichSu

#	Palette (Swing Controls)	Variable Name (đổi tên)	Thuộc tính cần đặt (Properties)	Cỡ gợi ý (R×C px)
1	Label	(giữ tên mặc định)	text = LỊCH SỬ THAO TÁC - BẢNG DBO.HISTORY font = Segoe UI 18 Bold	760×34
2	Table	tbl_history	(cột tạo bằng code trong constructor)	760×260
3	Button	btn_refresh	text = Tải lại	110×30
4	Label	lbl_status	text = Đang tải...	640×22



Hình 1.19. Form frm_LichSu khi đã có dữ liệu (6 lần Encrypt ở mục 3.6)

Bước 3. Bấm Ctrl+S, rồi nhấp phải tệp → Run File để xem thử: cửa sổ phải có bố cục giống Hình 1.19 (bấm nút lúc này chưa có tác dụng).

Bước 4. Mở tab Source, trong constructor thêm 3 dòng sau initComponents();:

Constructor của frm_LichSu

```
public frm_LichSu() {
    initComponents();
    setLocationRelativeTo(null);
    tbl_history.setModel(HistoryDao.emptyModel());
    loadData();
}
```

Dán vào tab Source, ngay SAU dấu } đóng của constructor frm_LichSu() (hàm loadData)

```
/** Đọc 50 dòng mới nhất từ SQL Server và hiện lên JTable. */
private void loadData() {
    try {
        tbl_history.setModel(HistoryDao.loadLatest(50));
        lbl_status.setText("Đã tải " + tbl_history.getRowCount() + " dòng mới nhất từ dbo.history.");
    } catch (java.sql.SQLException ex) {
        lbl_status.setText("Không đọc được lịch sử: " + ex.getMessage());
    }
}
```

Bước 5. Nhấp đúp nút Tải lại (btn_refresh) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_refreshActionPerformed

```
loadData();
```

Bước 6. Bấm Ctrl+S. Nếu NetBeans gạch đỏ dòng nào, rê chuột vào dòng đó để đọc lỗi và xem bảng lỗi ở cuối tài liệu.

Bước 7. Mở lại form frm_KetNoiSQL (tab Design) → nhấp đúp nút Xem lịch sử (btn_history) → dán đoạn mã sau vào giữa hai dấu { } của hàm btn_historyActionPerformed (đến đây frm_LichSu đã có nên không còn báo đỏ).

```
new frm_LichSu().setVisible(true);
```

Bước 8. Bấm Ctrl+S, rồi Shift+F11 (Clean and Build) → BUILD SUCCESS.

Nối hai nút còn lại vào frm_TongHop

Đến đây cả frm_KetNoiSQL và frm_LichSu đã có, nên dán hai đoạn mã dưới đây sẽ không còn báo đỏ.

Bước 1. Mở frm_TongHop ở tab Design → nhấp đúp nút Kết nối SQL Server (btn_sql) → dán đoạn mã sau vào giữa hai dấu { } của hàm btn_sqlActionPerformed.

```
new frm_KetNoiSQL().setVisible(true);
```

Bước 2. Nhấp đúp nút Xem lịch sử (btn_history) → dán đoạn mã sau vào giữa hai dấu { } của hàm btn_historyActionPerformed.

```
new frm_LichSu().setVisible(true);
```

Bước 3. Bấm Ctrl+S rồi Shift+F11 (Clean and Build) → BUILD SUCCESS.

3.6. Kiểm tra ghi và đọc lịch sử

Bước 1. Chạy frm_TongHop → bấm Kết nối SQL Server. Nhập Password của lab1_user (là LabUser@2026) (các ô khác giữ mặc định) → Test Connection.

Bước 2. Dòng trạng thái phải hiện Kết nối thành công: Microsoft SQL Server 16.00... - database LAB1_MaHoaCoDien.

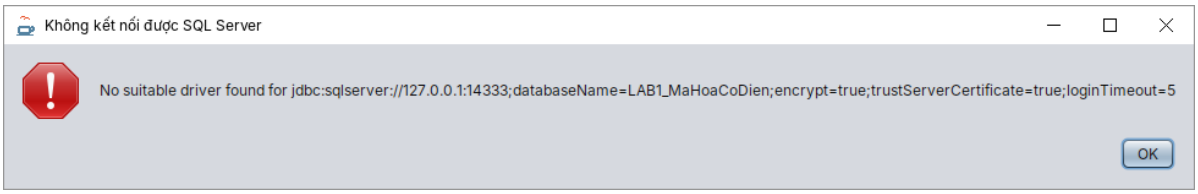
Bước 3. Quay lại frm_TongHop, mã hóa lần lượt 6 thuật toán (bảng ở mục 3.3). Dòng trạng thái hiện thêm Đã lưu lịch sử vào SQL Server.

Bước 4. Bấm **Xem lịch sử** → bảng hiện 6 dòng mới nhất (dòng mới ở trên).

Bước 5. Trong SSMS mở và chạy 03_Kiem_Tra_Du_Lieu.sql → thấy đúng 6 dòng đó. Chụp ảnh frm_LichSu và kết quả SSMS cho báo cáo.

03_Kiem_Tra_Du_Lieu.sql

```
USE [LAB1_MaHoaCoDien];
GO
SELECT TOP (20) id, created_at, algorithm, operation, input_text, result_text
FROM dbo.history ORDER BY id DESC;
GO
SELECT COUNT(*) AS so_ban_ghi FROM dbo.history;
GO
SELECT algorithm, COUNT(*) AS so_lan FROM dbo.history GROUP BY algorithm ORDER BY so_lan DESC;
GO
```



Hình 1.20. Ví dụ thông báo khi kết nối thất bại (ở đây do thiếu thư viện mssql-jdbc) – đọc dòng lỗi để sửa theo bảng lỗi

PHẦN 4. BÀI NÂNG CAO 02 – PHÂN TÍCH (TẤN CÔNG) MẬT MÃ CỔ ĐIỂN

A. Ba kỹ thuật sẽ cài đặt

Kỹ thuật	Ý tưởng	Áp dụng cho
Vét cạn (brute force)	Caesar chỉ có 26 khóa → thử hết, chọn bản rõ có nghĩa. Chương trình chấm điểm "giống tiếng Anh" bằng tần suất chữ cái để tự đánh dấu.	Caesar
Phân tích tần suất	Đếm số lần mỗi chữ xuất hiện. Trong tiếng Anh E, T, A, O xuất hiện nhiều nhất; chữ xuất hiện nhiều nhất trong bản mã thường là ảnh của E.	Caesar, mã thay thế đơn
Biết trước bản rõ (known plaintext)	Biết một cặp P/C cùng vị trí → khóa $K_i = (C_i - P_i) \bmod 26$.	Vigenère, Autokey

B. Tạo lớp AttackDemos

Bước 1. Nhấp phải package week_01 → New → Java Class... (nếu không thấy, chọn New → Other... → Java → Java Class).

Bước 2. Ô Class Name gõ AttackDemos → Finish. NetBeans mở tệp AttackDemos.java.

Bước 3. Bấm Ctrl+A, Delete để xóa nội dung mẫu, dán toàn bộ đoạn mã dưới đây, bấm Ctrl+S.

AttackDemos.java

```
package week_01;

import java.util.Locale;

/** Bài nâng cao 02: ba kỹ thuật phân tích mật mã cổ điển (chỉ dùng cho dữ liệu học tập). */
public final class AttackDemos {

    /** Tần suất (%) chữ A..Z trong văn bản tiếng Anh, dùng để chấm điểm các bản rõ thử. */
    private static final double[] ENGLISH = {8.2, 1.5, 2.8, 4.3, 12.7, 2.2, 2.0, 6.1, 7.0, 0.15, 0.77, 4.0, 2.4,
        6.7, 7.5, 1.9, 0.095, 6.0, 6.3, 9.1, 2.8, 0.98, 2.4, 0.15, 2.0, 0.074};

    private AttackDemos() {
    }

    /** 1) Vét cạn: thử đủ 26 khóa Caesar, đánh dấu khóa cho bản rõ "giống tiếng Anh" nhất. */
    public static String caesarBruteForce(String cipher) {
        int best = 0;
        double bestScore = -Double.MAX_VALUE;
        for (int k = 0; k < 26; k++) {
            double score = englishScore(CaesarCipher.decrypt(cipher, k));
            if (score > bestScore) {
                bestScore = score;
                best = k;
            }
        }
        StringBuilder sb = new StringBuilder("k  | Bản rõ thử\n");
        for (int k = 0; k < 26; k++) {
            sb.append(String.format("%2d | %s%s%n", k, CaesarCipher.decrypt(cipher, k), k == best ? "   <== có nghĩa nhất" :
""));
        }
        sb.append("Kết luận: khóa k = ").append(best).append(", bản rõ = ").append(CaesarCipher.decrypt(cipher, best));
        return sb.toString();
    }

    /** 2) Thống kê tần suất các chữ cái A-Z có trong bản mã (sắp theo bảng chữ cái). */
    public static String frequency(String cipher) {
```

```
        int[] count = new int[26];
        int total = 0;
        for (char c : cipher.toUpperCase(Locale.ROOT).toCharArray()) {
            if (c >= 'A' && c <= 'Z') {
                count[c - 'A']++;
                total++;
            }
        }
        if (total == 0) {
            return "Bản mã không có chữ cái A-Z.";
        }
        StringBuilder sb = new StringBuilder("Chữ | Số lần | Tần suất\n");
        for (int i = 0; i < 26; i++) {
            if (count[i] > 0) {
                sb.append(String.format(Locale.ROOT, " %c  |   %2d   | %5.1f %%%n", (char) ('A' + i), count[i], 100.0 * count[i] /
total));
            }
        }
        sb.append("Tổng số chữ: ").append(total);
        return sb.toString();
    }

    /** 3) Biết một cặp bản rõ/bản mã Vigenère: khóa_i = (C_i - P_i) mod 26. */
    public static String knownPlaintext(String plain, String cipher) {
        String p = plain.trim().toUpperCase(Locale.ROOT);
        String c = cipher.trim().toUpperCase(Locale.ROOT);
        if (p.isEmpty() || p.length() != c.length()) {
            throw new IllegalArgumentException("Bản rõ và bản mã phải cùng độ dài.");
        }
        if (!p.matches("[A-Z]+") || !c.matches("[A-Z]+")) {
            throw new IllegalArgumentException("Chỉ nhập chữ A-Z, không dấu, không khoảng trắng.");
        }
        StringBuilder key = new StringBuilder();
        for (int i = 0; i < p.length(); i++) {
            key.append((char) ('A' + Math.floorMod(c.charAt(i) - p.charAt(i), 26)));
        }
        return key.toString();
    }

    /** Điểm "giống tiếng Anh": cộng log tần suất của từng chữ; điểm càng LỚN càng giống tiếng Anh. */
    private static double englishScore(String text) {
        double score = 0;
        for (char ch : text.toUpperCase(Locale.ROOT).toCharArray()) {
            if (ch >= 'A' && ch <= 'Z') {
                score += Math.log(ENGLISH[ch - 'A']);
            }
        }
        return score;
    }
}
```

C. Thiết kế form frm_TanCong

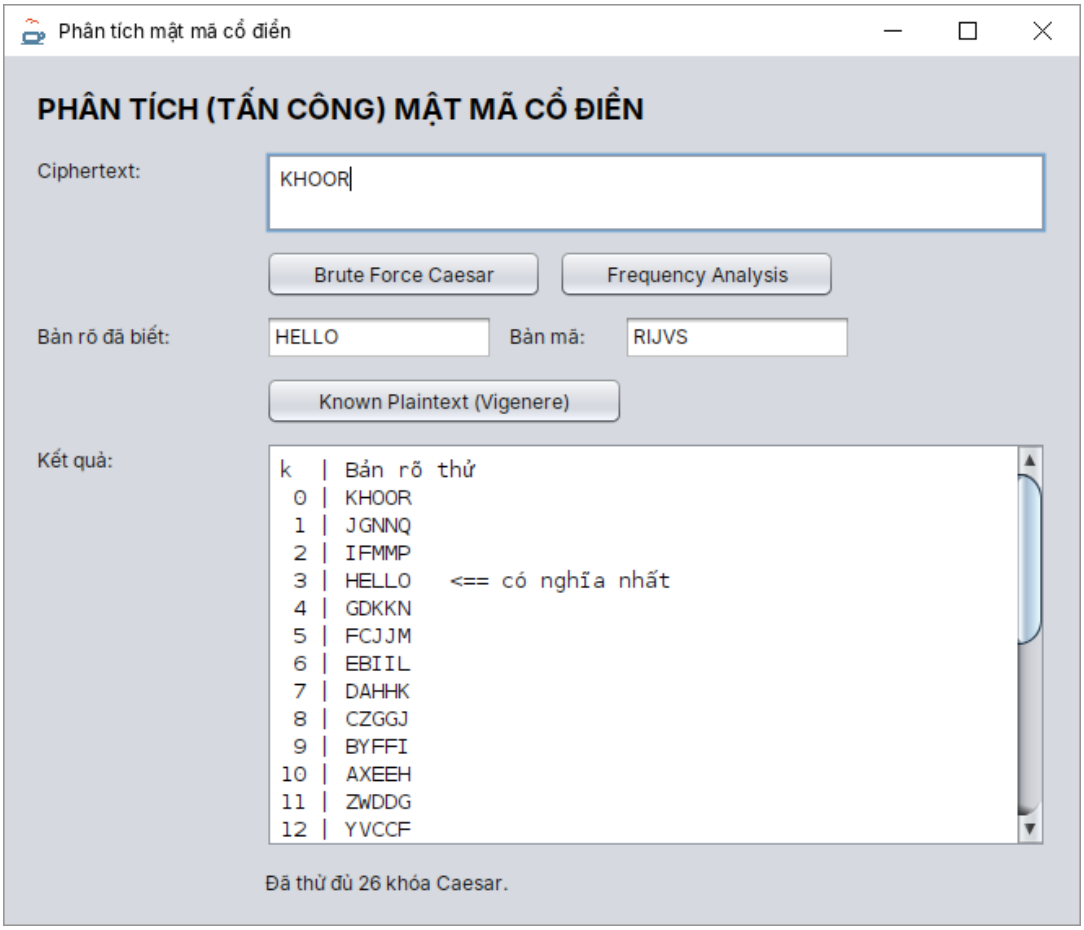
Bước 1. Tạo JFrame Form tên frm_TanCong (thao tác A). Đặt title = Phân tích mật mã cổ điển, defaultCloseOperation = DISPOSE.

Bước 2. Đặt các điều khiển theo thứ tự trong bảng (từ trên xuống, trái sang phải), đổi tên biến và đặt thuộc tính đúng như bảng.

Bảng 1.15. Điều khiển trên form frm_TanCong

#	Palette (Swing Controls)	Variable Name (đổi tên)	Thuộc tính cần đặt (Properties)	Cỡ gợi ý (R×C px)
1	Label	(giữ tên mặc định)	text = PHÂN TÍCH (TẤN CÔNG) MẬT MÃ CỔ ĐIỂN font = Segoe UI 18 Bold	620×34
2	Label	(giữ tên mặc định)	text = Ciphertext:	130×22
3	Text Area	txt_cipher	lineWrap = tích rows = 2	480×50

#	Palette (Swing Controls)	Variable Name (đổi tên)	Thuộc tính cần đặt (Properties)	Cỡ gợi ý (R×C px)
4	Button	btn_bruteforce	text = Brute Force Caesar	170×30
5	Button	btn_frequency	text = Frequency Analysis	170×30
6	Label	(giữ tên mặc định)	text = Bản rõ đã biết:	130×22
7	Text Field	txt_known_plain	text = HELLO	140×28
8	Label	(giữ tên mặc định)	text = Bản mã:	60×22
9	Text Field	txt_known_cipher	text = RIJVS	140×28
10	Button	btn_known	text = Known Plaintext (Vigenere)	220×30
11	Label	(giữ tên mặc định)	text = Kết quả:	130×22
12	Text Area	txt_result	lineWrap = không tích rows = 3 editable = bỏ tích	480×250
13	Label	lbl_status	text = Sẵn sàng.	480×22



Hình 1.21. frm_TanCong sau khi bấm Brute Force Caesar với KHOOR

Bước 3. Bấm Ctrl+S, rồi nhấp phải tệp → Run File để xem thử: cửa sổ phải giống Hình 1.21 (bấm nút lúc này chưa có tác dụng).

Bước 4. Thêm import javax.swing.JOptionPane; và setLocationRelativeTo(null);.

Bước 5.Nhấp đúp nút Brute Force Caesar (btn_bruteforce) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_bruteforceActionPerformed

```
String cipher = txt_cipher.getText().trim();
if (cipher.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Hãy nhập Ciphertext.");
    return;
}
txt_result.setText(AttackDemos.caesarBruteForce(cipher));
txt_result.setCaretPosition(0);
lbl_status.setText("Đã thử đủ 26 khóa Caesar.");
```

Bước 6.Nhấp đúp nút Frequency Analysis (btn_frequency) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_frequencyActionPerformed

```
String cipher = txt_cipher.getText().trim();
if (cipher.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Hãy nhập Ciphertext.");
    return;
}
txt_result.setText(AttackDemos.frequency(cipher));
txt_result.setCaretPosition(0);
lbl_status.setText("Đã thống kê tần suất chữ cái.");
```

Bước 7.Nhấp đúp nút Known Plaintext (Vigenere) (btn_known) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_knownActionPerformed

```
try {
    String key = AttackDemos.knownPlaintext(txt_known_plain.getText(), txt_known_cipher.getText());
    txt_result.setText("Khóa Vigenère suy ra từng vị trí: " + key
            + "\nNếu khóa ngắn hơn bản rõ, khóa sẽ lặp lại (KEY KEY ...).");
    lbl_status.setText("Khóa suy ra: " + key);
} catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(),
            "Dữ liệu không hợp lệ", JOptionPane.ERROR_MESSAGE);
}
```

Bước 8.Bấm Ctrl+S. Nếu NetBeans gạch đỏ dòng nào, rê chuột vào dòng đó để đọc lỗi và xem bảng lỗi ở cuối tài liệu.

D. Các thí nghiệm bắt buộc (ghi kết quả vào báo cáo)

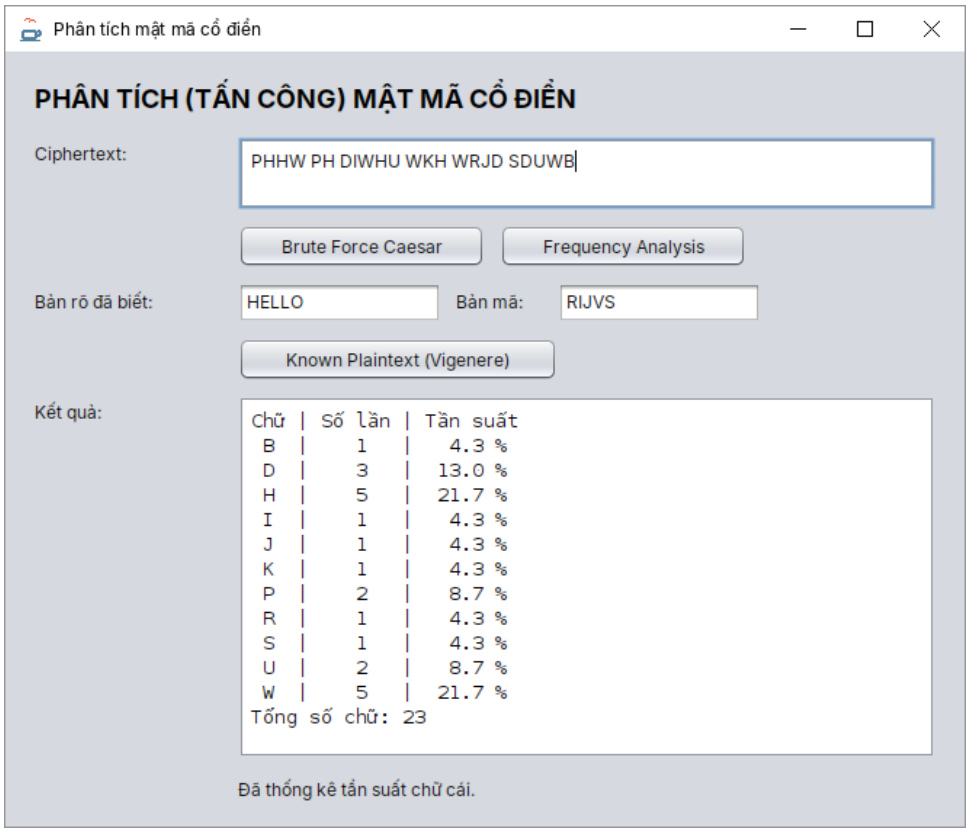
Bước 1.Ô Ciphertext nhập KHOOR → Brute Force Caesar. Kết quả có 26 dòng, dòng k = 3 là HELLO được đánh dấu; dòng cuối:

```
k  | Bản rõ thử
 3 | HELLO   <== có nghĩa nhất
...
Kết luận: khóa k = 3, bản rõ = HELLO
```

Bước 2. Ô Ciphertext nhập PHHW PH DIWHU WKH WRJD SDUWB (bản mã Caesar k=3 của "MEET ME AFTER THE TOGA PARTY") → Frequency Analysis.

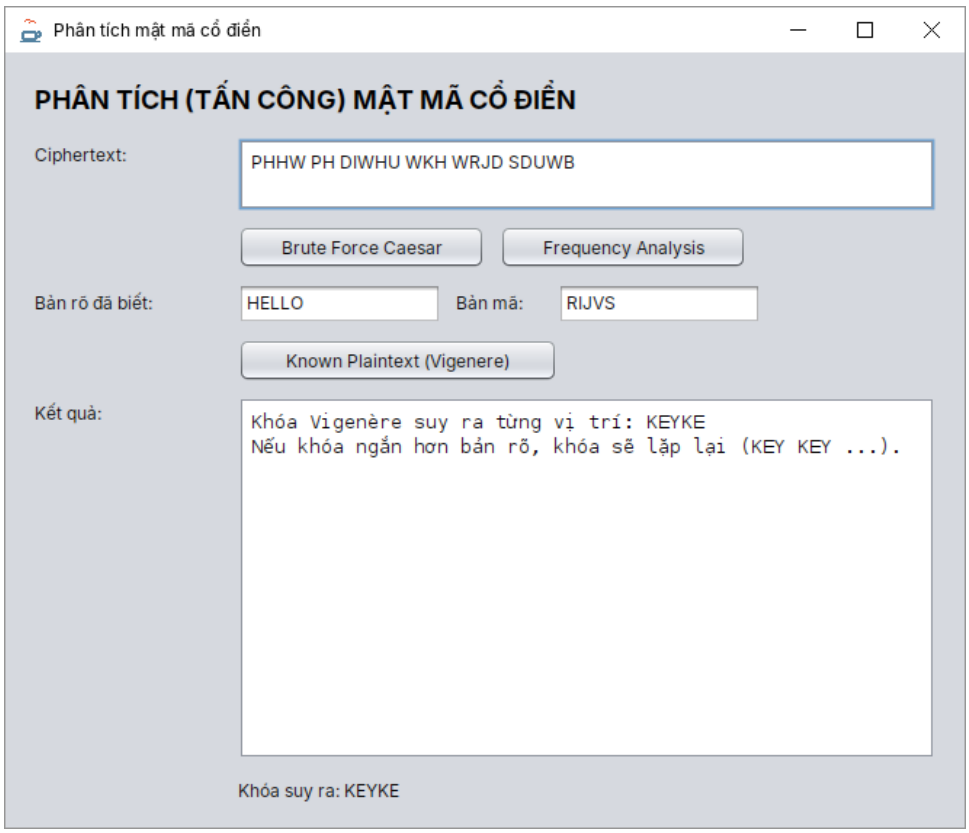
```
Chữ | Số lần | Tần suất
 B |   1   |  4.3 %
 D |   3   | 13.0 %
 H |   5   | 21.7 %
 I |   1   |  4.3 %
 J |   1   |  4.3 %
 K |   1   |  4.3 %
 P |   2   |  8.7 %
 R |   1   |  4.3 %
 S |   1   |  4.3 %
 U |   2   |  8.7 %
 W |   5   | 21.7 %
Tổng số chữ: 23
```

Nhận xét: H và W xuất hiện nhiều nhất (21,7%). Giả sử H là ảnh của E: H − E = 7 − 4 = **3** → đúng khóa k = 3. Với bản mã quá ngắn (như KHOOR) thống kê tần suất không đáng tin.



Hình 1.22. Kết quả Frequency Analysis

Bước 3. Ô Bản rõ đã biết HELLO, ô Bản mã RIJVS → Known Plaintext (Vigenere) → kết quả KEYKE (khóa KEY lặp lại: K E Y K E).



Hình 1.23. Suy ra khóa Vigenère từ một cặp bản rõ/bản mã

E. Nhận xét cần viết trong báo cáo

Thuật toán	Điểm yếu chính	Cách tấn công
Caesar	Chỉ 26 khóa	Vét cạn; phân tích tần suất
Vigenère	Khóa lặp lại theo chu kỳ	Tìm độ dài khóa (Kasiski/Friedman) rồi phân tích tần suất từng cột; known plaintext
Rail Fence	Số hàng nhỏ, không đổi chữ cái	Thử mọi số hàng 2..n; tần suất chữ giữ nguyên như bản rõ
Playfair	Thay thế theo cặp, 25 chữ	Thống kê cặp chữ (digram); known plaintext dựng lại ma trận
Transposition	Chỉ đổi vị trí, tần suất giữ nguyên	Thử độ dài khóa nhỏ, sắp xếp cột để ra từ có nghĩa
Autokey	Khóa chứa chính bản rõ	Đoán từ thông dụng (THE, AND) để dò dần khóa; known plaintext

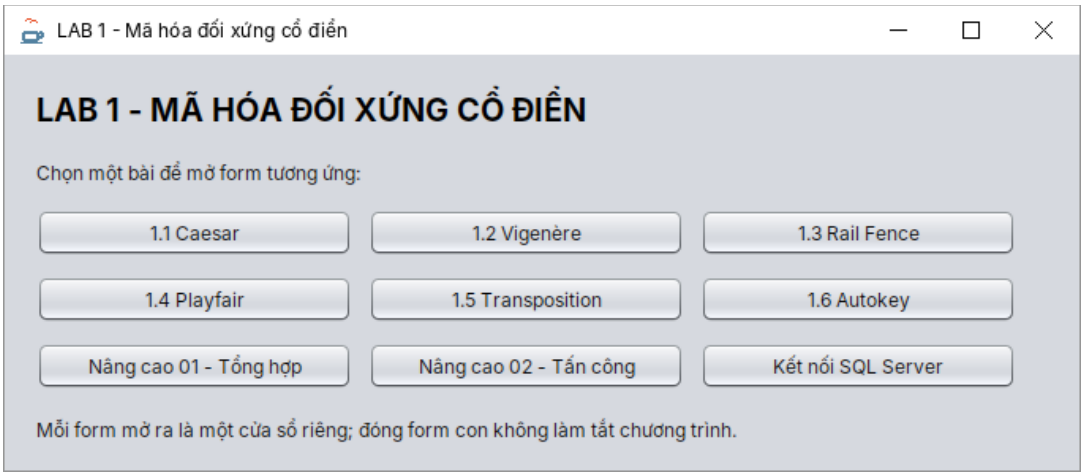
PHẦN 5. TẠO MENU CHÍNH VÀ CHẠY TOÀN BỘ CHƯƠNG TRÌNH

Bước 1. Tạo JFrame Form tên frm_Menu (thao tác A). Đặt title = LAB 1 - Mã hóa đối xứng cổ điển, defaultCloseOperation = DISPOSE (riêng frm_Menu giữ EXIT_ON_CLOSE).

Bước 2. Đặt các điều khiển theo thứ tự trong bảng (từ trên xuống, trái sang phải), đổi tên biến và đặt thuộc tính đúng như bảng.

Bảng 1.16. Điều khiển trên form frm_Menu

#	Palette (Swing Controls)	Variable Name (đổi tên)	Thuộc tính cần đặt (Properties)	Cỡ gợi ý (R×C px)
1	Label	(giữ tên mặc định)	text = LAB 1 - MÃ HÓA ĐỐI XỨNG CỔ ĐIỂN font = Segoe UI 20 Bold	640×38
2	Label	(giữ tên mặc định)	text = Chọn một bài để mở form tương ứng:	640×22
3	Button	btn_caesar	text = 1.1 Caesar	200×30
4	Button	btn_vigenere	text = 1.2 Vigenère	200×30
5	Button	btn_railfence	text = 1.3 Rail Fence	200×30
6	Button	btn_playfair	text = 1.4 Playfair	200×30
7	Button	btn_transposition	text = 1.5 Transposition	200×30
8	Button	btn_autokey	text = 1.6 Autokey	200×30
9	Button	btn_tonghop	text = Nâng cao 01 - Tổng hợp	200×30
10	Button	btn_tancong	text = Nâng cao 02 - Tấn công	200×30
11	Button	btn_sql	text = Kết nối SQL Server	200×30
12	Label	(giữ tên mặc định)	text = Mỗi form mở ra là một cửa sổ riêng; đóng form con không làm tắt chương trình.	640×22



Hình 1.24. Form frm_Menu sau khi thiết kế (chạy thử bằng Shift+F6)

Bước 3. Bấm Ctrl+S, rồi nhấp phải tệp → Run File để xem thử: cửa sổ phải giống Hình 1.24 (bấm nút lúc này chưa có tác dụng).

Bước 4. Nhấp đúp nút 1.1 Caesar (btn_caesar) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_caesarActionPerformed

```
new frm_Caesar().setVisible(true);
```

Bước 5. Nhấp đúp nút 1.2 Vigenère (btn_vigenere) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_vigenereActionPerformed

```
new frm_Vigenere().setVisible(true);
```

Bước 6. Nhấp đúp nút 1.3 Rail Fence (btn_railfence) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_railfenceActionPerformed

```
new frm_RailFence().setVisible(true);
```

Bước 7. Nhấp đúp nút 1.4 Playfair (btn_playfair) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_playfairActionPerformed

```
new frm_Playfair().setVisible(true);
```

Bước 8. Nhấp đúp nút 1.5 Transposition (btn_transposition) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_transpositionActionPerformed

```
new frm_Transposition().setVisible(true);
```

Bước 9. Nhấp đúp nút 1.6 Autokey (btn_autokey) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_autokeyActionPerformed (

```
new frm_Autokey().setVisible(true);
```

Bước 10. Nhấp đúp nút Nâng cao 01 - Tổng hợp (btn_tonghop) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_tonghopActionPerformed

```
new frm_TongHop().setVisible(true);
```

Bước 11. Nhấp đúp nút Nâng cao 02 - Tấn công (btn_tancong) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_tancongActionPerformed

```
new frm_TanCong().setVisible(true);
```

Bước 12. Nhấp đúp nút Kết nối SQL Server (btn_sql) → dán đoạn mã sau vào GIỮA hai dấu { } của hàm btn_sqlActionPerformed

```
new frm_KetNoiSQL().setVisible(true);
```

Bước 13. Bấm Ctrl+S. Nếu NetBeans gạch đỏ dòng nào, rê chuột vào dòng đó để đọc lỗi và xem bảng lỗi ở cuối tài liệu.

Bước 14. Nhấp phải tên project → Run (hoặc bấm F6). Cửa sổ frm_Menu phải mở ra; bấm thử từng nút, mỗi nút mở đúng form tương ứng.